IT5311

PROGRAMMING AND DATA STRUCUTURES LABORATORY

A PRACTICAL RECORD

Submitted by

Yuvaraj K
2022506079

B. Tech IT (3/8)

DEPARTMENT OF INFORMATION TECHNOLOGY
MADRAS INSTITUTE OF TECHNOLOGY,
ANNA UNIVERSITY, CHENNAI – 600 044

December 2023

BONAFIDE CERTIFICATE

Name : Yuvaraj K

Register Number : 2022506079

Subject : IT5311 Programming and Data Structures Laboratory

Department : Information Technology

Certified to the bonafide record of practical work done by Mr. Yuvaraj K in the **IT5311 Programming and Data Structures Laboratory** during the period August 2023 to December 2023.

Date:

Dr. Geetha Ramani
STAFF-IN-CHARGE

Submitted for the Practical Examination held on:

Internal Examiner **External Examiner**

INDEX

EXP. NO	DATE	TITLE	PAGE	SIGNATURE
1	07/09/23	FUNDAMENTALS OF C PROGRAMMING - I	1	
2	14/09/23	FUNDAMENTALS OF C PROGRAMMING – II	24	
3	21/09/23	IMPLEMENTATION OF SINGLE DIMENSIONAL, MULTI-DIMENSIONAL ARRAYS AND FUNCTIONS	59	
4	05/10/23	IMPLEMENTATION OF SEARCHING AND SORTING TECHNIQUES	93	
5	12/10/23	IMPLEMENTATION OF STRUCTURE, UNION, AND ENUMERATION DATA TYPES	102	
6	2/11/23	IMPLEMENTATION OF DYNAMIC MEMORY ALLOCATION AND FILE HANDLING	112	
7	16/11/23	IMPLEMENTATION OF LINKED LIST	116	
8	23/11/23	IMPLEMENTATION OF STACK USING ARRAY AND LINKED LIST	128	
9	30/11/23	IMPLEMENTATION OF QUEUE USING ARRAY AND LINKED LIST	145	
10	14/12/23	IMPLEMENTATION OF BINARY SEARCH TREE AND TREE TRAVERSALS	160	
11	21/12/23	IMPLEMENTATION OF HEAP SORT	172	

Exp. No: 1
Date: 07/09/23

FUNDAMENTALS OF C PROGRAMMING - I

AIM

To explore the fundamentals of C programming.

INSTRUCTIONS TO WRITE C PROGRAMS IN CENT OS TERMINAL

(i) Right click on Desktop
(ii) Click *'Open Terminal'*
(iii) In the terminal

a.type *'vi filename.c'*
b.press *'i'* to edit
c.type your c program
d.press *'esc'*
e.type *':wq'* to write and quit, *':q!'* to quit

(iv) type *'cc filename.c'* to compile the program
(v) type *'./a.out'* to run the compiled program

HELLO WORLD PROGRAM

```
#include <stdio.h>
void main() {
	printf("Hello World\n");
}
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/c_programs/exp_0$ cc helloWorld.c
warspinix@Yuvaraj-Laptop:~/c_programs/exp_0$ ./a.out
Hello World
warspinix@Yuvaraj-Laptop:~/c_programs/exp_0$
```

SAME PROGRAM, BUT WITH *int* RETURN TYPE

```
#include <stdio.h>
int main() {
        printf("Hello World\n");
        return 0;
}
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/c_programs/exp_0$ cc helloWorld.c
warspinix@Yuvaraj-Laptop:~/c_programs/exp_0$ ./a.out
Hello World
warspinix@Yuvaraj-Laptop:~/c_programs/exp_0$
```

The keyword in front of the function tells its return datatype. If it is void, nothing is to be returned. If it is int, some integer has to be returned.

MY PROFILE (USING MULTIPLE *print* STATEMENTS)

```
#include <stdio.h>
int main() {
    printf("Name: Yuvaraj\nDOB: 21/09/2004\n");
    printf("Age: 18\nCollege: Madras Institute of Technology\n");
    printf("Branch and Department: B.Tech Information Technology\nYear: 2\n");
    return 0;
}
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/c_programs/exp_0$ cc profile.c
warspinix@Yuvaraj-Laptop:~/c_programs/exp_0$ ./a.out
Name: Yuvaraj
DOB: 21/09/2004
Age: 18
College: Madras Institute of Technology
Branch and Department: B.Tech Information Technology
Year: 2
warspinix@Yuvaraj-Laptop:~/c_programs/exp_0$
```

VARIABLES IN C

```
#include <stdio.h>
int main() {

        int a,b,c;
        float f;

        a=10;
        b=20;
        c=a+b;
        printf("Value of C: %d\n",c);

        f=70.0/3.0;
        printf("Value of F: %f\n",f);
        return 0;
}
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/c_programs/exp_01$ cc arithmetic.c
warspinix@Yuvaraj-Laptop:~/c_programs/exp_01$ ./a.out
Value of C: 30
Value of F: 23.333334
warspinix@Yuvaraj-Laptop:~/c_programs/exp_01$
```

ESCAPE SEQUENCES

```
#include <stdio.h>
int main() {
	printf("Hello\tWorld\n\t\tHello\\\"World\"\n");
	return 0;
}
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/c_programs/exp_01$ cc escape_sequence.c
warspinix@Yuvaraj-Laptop:~/c_programs/exp_01$ ./a.out
Hello	World
		Hello\"World"
warspinix@Yuvaraj-Laptop:~/c_programs/exp_01$
```

BASIC FUNCTION DECLARATION, DEFINITION AND CALL

```
int func();

#include <stdio.h>
int main() {
	int i=func();
	printf("Value of i = %d\n",i);
	return 0;
}

int func() {
	return 0;
}
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/c_programs/exp_01$ cc function.c
warspinix@Yuvaraj-Laptop:~/c_programs/exp_01$ ./a.out
Value of i = 0
warspinix@Yuvaraj-Laptop:~/c_programs/exp_01$
```

#define PREPROCESSOR

```
#include <stdio.h>
#define l 10
#define w 5
int main() {
        int area;
        area=l*w;
        printf("Area = %d\n",area);

        return 0;
}
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/c_programs/exp_01$ cc define.c
warspinix@Yuvaraj-Laptop:~/c_programs/exp_01$ ./a.out
Area = 50
warspinix@Yuvaraj-Laptop:~/c_programs/exp_01$
```

const KEYWORD

```
#include <stdio.h>

int main() {
    const int l=10;
    const int width=5;
    int area;
    area=l*w;
    printf("Area = %d\n",area);
    return 0;
}
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/c_programs/exp_01$ cc constant.c
warspinix@Yuvaraj-Laptop:~/c_programs/exp_01$ ./a.out
Area = 50
warspinix@Yuvaraj-Laptop:~/c_programs/exp_01$
```

if STATEMENT

```
#include <stdio.h>
int main() {
	int a=10;
	if (a<20)
		printf("a is less than 20\n");
	printf("Value of a = %d\n",a);
	return 0;
}
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/c_programs/exp_01$ cc if.c
warspinix@Yuvaraj-Laptop:~/c_programs/exp_01$ ./a.out
a is less than 20
Value of a = 10
warspinix@Yuvaraj-Laptop:~/c_programs/exp_01$
```

if… else STATEMENT

```
#include <stdio.h>
int main() {
	int a = 100;
	if(a<20)
		printf("a is less than 20.\n");
	else
		printf("a is not less than 20.\n");
	printf("Value of a = %d\n",a);
	return 0;
}
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/c_programs/exp_01$ cc if_else.c
warspinix@Yuvaraj-Laptop:~/c_programs/exp_01$ ./a.out
a is not less than 20.
Value of a = 100
warspinix@Yuvaraj-Laptop:~/c_programs/exp_01$
```

NESTED *if* STATEMENT

```
#include <stdio.h>
int main() {
	int a=100;
	int b=200;
	if (a==100) {
		if (b==200)
			printf("Value of a is 100 and Value of b is 200.\n");
		else
			printf("Value of a is 100 but Value of b is not 200.\n");
	} else {
		printf("Value of a is not 100.\n");
	}
	printf("a=%d\n",a);
	printf("b=%d\n",b);
	return 0;
}
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/c_programs/exp_01$ cc nested_if.c
warspinix@Yuvaraj-Laptop:~/c_programs/exp_01$ ./a.out
Value of a is 100 and Value of b is 200.
a=100
b=200
warspinix@Yuvaraj-Laptop:~/c_programs/exp_01$
```

In a nested *if* statement, we will have an *if* statement within an *if* statement. If the condition in the outer *if* is true, the compiler will check the inner *if*. We can have a combination of how many ever *if, else if, and else* statements we want, nested with proper syntax in order to suit our needs.

In the above program, integer variable *a* has been assigned the variable 100, and variable *b* has been assigned the variable 200. The first condition checks whether *a* is equal to 100, which is true. So the statements within the *if* block are compiled. We have a nested *if* block, where the first condition is whether *b* is equal to 200. This is also true, so the statement within this is executed.

switch STATEMENT

```
#include <stdio.h>
int main() {
	char grade='B';
	switch (grade) {
		case 'A':
			printf("Excellent!\n");
			break;
		case 'B':
			printf("Very Good!\n");
			break;
		case 'C':
			printf("Good!\n");
			break;
		case 'D':
			printf("You passed.\n");
			break;
		case 'E':
			printf("Better try again.\n");
			break;
		default:
			printf("Invalid Grade.\n");
	}
	printf("Your grade is %d\n",grade);
	return 0;
}
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/c_programs/exp_01$ cc switch.c
warspinix@Yuvaraj-Laptop:~/c_programs/exp_01$ ./a.out
Very Good!
Your grade is B.
warspinix@Yuvaraj-Laptop:~/c_programs/exp_01$
```

The *switch* statement is useful when checking multiple *cases*, and evaluating based on the matching case. Here, *grade* is ‘B’. So, the second *case* is successful, and the corresponding *printf* statement is executed. The compiler executes every statement after a successful *case* until it sees a *break.* For example, if we remove the *break* in *case* ‘B’, the *printf* statements in both cases ‘B’ and ‘C’ will be executed. If none of the cases match, the statements within *default* are executed.

NESTED *switch* STATEMENT

```
#include <stdio.h>
int main() {
	int a=100;
	int b=200;
	switch (a) {
		case 100:
			printf("This is part of outer switch.\n");
			switch (b) {
				case 200:
					printf("This is part of inner switch.\n");
			}
	}
	return 0;
}
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/c_programs/exp_01$ cc nested_switch.c
warspinix@Yuvaraj-Laptop:~/c_programs/exp_01$ ./a.out
This is part of outer switch.
This is part of inner switch.
warspinix@Yuvaraj-Laptop:~/c_programs/exp_01$
```

TERNARY OPERATOR

```
#include <stdio.h>
int main() {
	int a,b,c;
	a=1,b=2;
	a>b ? printf("%d\n",a): printf("%d\n",b);
	return 0;
}
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/c_programs/exp_01$ cc ternary.c
warspinix@Yuvaraj-Laptop:~/c_programs/exp_01$ ./a.out
2
warspinix@Yuvaraj-Laptop:~/c_programs/exp_01$
```

while LOOP

```
#include <stdio.h>
int main() {
	int a=10;
	while (a<20) {
		printf("Value of a = %d\n",a);
		a++;
	}
	   return 0;
}
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/c_programs/exp_01$ cc while.c
warspinix@Yuvaraj-Laptop:~/c_programs/exp_01$ ./a.out
Value of a = 10
Value of a = 11
Value of a = 12
Value of a = 13
Value of a = 14
Value of a = 15
Value of a = 16
Value of a = 17
Value of a = 18
Value of a = 19
warspinix@Yuvaraj-Laptop:~/c_programs/exp_01$
```

In *while* loop, the initialisation is done beforehand. The condition is checked after the *while* keyword is introduced, and the increment/decrement/update is done inside the *while* block. If the condition is false initially, the *while* block is never entered.

for LOOP

```
#include <stdio.h>
int main() {
	int a;
	for (a=10; a<20; a++) {
		printf("Value of a = %d\n",a);
	}
	return 0;
}
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/c_programs/exp_01$ cc for.c
warspinix@Yuvaraj-Laptop:~/c_programs/exp_01$ ./a.out
Value of a = 10
Value of a = 11
Value of a = 12
Value of a = 13
Value of a = 14
Value of a = 15
Value of a = 16
Value of a = 17
Value of a = 18
Value of a = 19
warspinix@Yuvaraj-Laptop:~/c_programs/exp_01$
```

In *for* loop, the initialisation, condition check, and increment/decrement are done in the same line. Similar to *while* loop, if the condition is false initially, the *for* block is never entered.

do while LOOP

```
#include <stdio.h>
int main() {
	int a=10;
	do {
		printf("Value of a = %d\n",a);
		a++;
	} while (a<20);
	return 0;
}
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/c_programs/exp_01$ cc do_while.c
warspinix@Yuvaraj-Laptop:~/c_programs/exp_01$ ./a.out
Value of a = 10
Value of a = 11
Value of a = 12
Value of a = 13
Value of a = 14
Value of a = 15
Value of a = 16
Value of a = 17
Value of a = 18
Value of a = 19
warspinix@Yuvaraj-Laptop:~/c_programs/exp_01$
```

In the case of *do while*, the code block is always executed at least once, since the condition is checked only after the block.

All these different looping constructs come in extremely handy for repetitive tasks. The choice of which style of looping to be used depends on the programmer, and also the problem at hand.

NESTED LOOP

```
#include <stdio.h>
int main() {
	int i,j;
	for (i=2;i<100;i++) {
		for (j=2;j<=(i/j);j++) {
			if (!(i%j))
				break;
		}
		if (j>(i/j))
			printf("%d is prime\n",i);
	}
	return 0;
}
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/c_programs/exp_01$ cc nested_loops.c
warspinix@Yuvaraj-Laptop:~/c_programs/exp_01$ ./a.out
2 is prime
3 is prime
5 is prime
7 is prime
11 is prime
13 is prime
17 is prime
19 is prime
23 is prime
29 is prime
31 is prime
37 is prime
41 is prime
43 is prime
47 is prime
53 is prime
59 is prime
61 is prime
67 is prime
71 is prime
73 is prime
79 is prime
83 is prime
89 is prime
97 is prime
warspinix@Yuvaraj-Laptop:~/c_programs/exp_01$
```

Similar to nested *if*, we can also have nested loops, that is, a loop within a loop. In the above program, a for loop is nested within another for loop. The outer loop cycles from the numbers 2 to 99, and the inner loop checks if these numbers are prime. If they are indeed prime numbers, the corresponding *printf* statement is executed. So, this program gives us all the prime numbers from 1 to 100.

break STATEMENT

```
#include <stdio.h>
int main() {
	int a=10;
	while (a<20) {
		printf("Value of a = %d\n",a);
		a++;
		if (a>15)
			break;
	}
	return 0;
}
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/c_programs/exp_01$ cc break.c
warspinix@Yuvaraj-Laptop:~/c_programs/exp_01$ ./a.out
Value of a = 10
Value of a = 11
Value of a = 12
Value of a = 13
Value of a = 14
Value of a = 15
warspinix@Yuvaraj-Laptop:~/c_programs/exp_01$
```

The *break* statement is a jump statement that exits the innermost loop. The control flow of the program is passed to the statement that comes immediately after the loop.

In the above program, the variable *a* is cycled from the value 10 to 19. Each value is displayed to the standard output. However, if the value is 15, the *break* statement is executed. This means that the loop will exit at this point and the compiler will go to the statement that comes after the loop.

continue STATEMENT

```
#include <stdio.h>
int main() {
	int a=10;
	do {
		if (a==15) {
			a++;
			continue;
		}

		printf("Value of a = %d\n",a);
		a++;
	} while (a<20);
	return 0;
}
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/c_programs/exp_01$ cc continue.c
warspinix@Yuvaraj-Laptop:~/c_programs/exp_01$ ./a.out
Value of a = 10
Value of a = 11
Value of a = 12
Value of a = 13
Value of a = 14
Value of a = 16
Value of a = 17
Value of a = 18
Value of a = 19
warspinix@Yuvaraj-Laptop:~/c_programs/exp_01$
```

goto STATEMENT

```
#include <stdio.h>
int main() {
        int a=10;
        LOOP: do {
                if (a==15) {
                        a++;
                        goto LOOP;
                }
                printf("Value of a = %d\n",a);
                a++;
        } while (a<20);
        return 0;
}
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/c_programs/exp_01$ cc goto.c
warspinix@Yuvaraj-Laptop:~/c_programs/exp_01$ ./a.out
Value of a = 10
Value of a = 11
Value of a = 12
Value of a = 13
Value of a = 14
Value of a = 16
Value of a = 17
Value of a = 18
Value of a = 19
warspinix@Yuvaraj-Laptop:~/c_programs/exp_01$
```

The *goto* statement contains a *label*, and when this statement is executed, the control flow of the program goes to the *label*. In this program, the *label* has been named as 'LOOP'. When the value of *a* becomes 15, it is incremented, and the *goto* statement is executed, meaning the control flow is shifted, moving outside the loop. That is why the value of *a* as 15 is not printed, but all the others are.

INFINITE LOOP

```
#include <stdio.h>
int main() {
	for (;;)
		printf("This loop will run forever.\n");
	return 0;
}
```

OUTPUT

```
This loop will run forever.
This loop will run forever.
This loop will run forever.
This loop will run forever.
This loop will run forever.
This loop will run forever.
This loop will run forever.
This loop will run forever.
This loop will run forever.
This loop will run forever.
This loop will run forever.
This loop will run forever.
This loop will run forever.
This loop will run forever.
This loop will run forever.
This loop will run forever.
This loop will run forever.
This loop will run forever.
This loop will run forever.
This loop will run forever.
This loop will run forever.
This loop will run forever.
This loop will run forever.
This loop will run forever.
This loop will run forever.
This loop will run forever.
This loop will run forever.
This loop will run forever.
This loop will run forever.
This loop will run forever.
This loop will run forever.
This loop will run forever.
This loop will run forever.
This loop will run forever.
This loop will run forever.
This loop will run forever.
This loop will run forever.
This loop will run forever.
This loop will run forever.
This loop will run forever.
This loop will run forever.
This^C
warspinix@Yuvaraj-Laptop:~/c_programs/exp_01$
```

MAXIMUM OF 2 NUMBERS USING FUNCTION

```
#include <stdio.h>
int max(int x, int y) {
	if (x>y)
		return x;
	else
		return y;
}
int main() {
	int a=100, b=200;
	int m=max(a,b);
	printf("The larger number between %d and %d is is %d\n",a,b,m);
	return 0;
}
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/c_programs/exp_01$ cc maximum_using_function.c
warspinix@Yuvaraj-Laptop:~/c_programs/exp_01$ ./a.out
The larger number between 100 and 200 is is 200.
warspinix@Yuvaraj-Laptop:~/c_programs/exp_01$
```

CALL BY VALUE

```
void swap(int, int);
#include <stdio.h>
void main() {
	int a=100, b=200;
	printf("A and B before swap: %d and %d respectively\n",a,b);
	swap(a,b);
}
void swap(int x, int y) {
	int temp=x;
	x=y;
	y=temp;
	printf("A and B after swap: %d and %d respectively\n",x,y);
}
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/c_programs/exp_01$ cc call_by_value.c
warspinix@Yuvaraj-Laptop:~/c_programs/exp_01$ ./a.out
A and B before swap: 100 and 200 respectively
A and B after swap: 200 and 100 respectively
warspinix@Yuvaraj-Laptop:~/c_programs/exp_01$
```

CALL BY REFERENCE

```
#include <stdio.h>
void swap(int *, int *);
int main() {
	int a=100;
	int b=200;
	printf("A and B before swap: %d and %d respectively\n",a,b);
	swap(&a,&b);
	printf("A and B after swap: %d and %d respectively\n",a,b);
	return 0;
}
void swap(int *x, int *y) {
	int temp;
	temp=*x;
	*x=*y;
	*y=temp;
}
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/c_programs/exp_01$ cc call_by_reference.c
warspinix@Yuvaraj-Laptop:~/c_programs/exp_01$ ./a.out
A and B before swap: 100 and 200 respectively
A and B after swap: 200 and 100 respectively
warspinix@Yuvaraj-Laptop:~/c_programs/exp_01$
```

FACTORIAL OF A NUMBER

```
#include <stdio.h>
void main() {
	int n;
	printf("Enter a Number: ");
	scanf("%d",&n);

	int fact=1;
	int i;

	if (n<0) {
		printf("Factorial can only be found for positive numbers.\n");
	}
	else {
		if (n==0) {
			fact=1;
		}
		else {
			for (i=1; i<=n; i++) {
				fact*=i;
			}
		}
		printf("Factorial of %d is %d.",n,fact);
	}
}
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/c_programs/exp_0$ cc fact.c
warspinix@Yuvaraj-Laptop:~/c_programs/exp_0$ ./a.out
Enter a Number: 7
Factorial of 7 is 5040.
warspinix@Yuvaraj-Laptop:~/c_programs/exp_0$
```

FIRST *n* FIBONACCI NUMBERS

```
#include <stdio.h>
void main() {
	int n;
	printf("Enter a Number: ");
	scanf("%d",&n);
	if (n<=0)
		printf("Please enter a positive number.");
	else {
		printf("Fibonacci Series up to %d terms\n\n",n);
		if (n==1)
			printf("%d",0);
		else if (n==2)
			printf("%d %d",0,1);
		else {
			int a=0;
			int b=1;
			printf("%d %d ",a,b);
			int c;
			int i;
			for (i=0;i<(n-2);i++) {
				c=a+b;
				printf("%d ",c);
				a=b;
				b=c;
			}
		}
	}
	printf("\n");
}
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/c_programs/exp_0$ cc fibo.c
warspinix@Yuvaraj-Laptop:~/c_programs/exp_0$ ./a.out
Enter a Number: 11
Fibonacci Series up to 11 terms

0 1 1 2 3 5 8 13 21 34 55
warspinix@Yuvaraj-Laptop:~/c_programs/exp_0$
```

INTERPRETATION OF THE ABOVE CODE

#include <stdio.h>	It is a preprocessor directive statement. *<stdio.h>* is the header file that is included so that we can work with standard input and output functions.
main()	The code inside this function is what is compiled.
void	When placed in front of any function (such as *main()*), it indicates the return datatype, which in this case is void/null/nothing. We can also have int return type or float return type, etc.
int n	Variables have to be declared with their type in C. We can assign a value to the variable during declaration, or do so at a later part of the program.
printf	Output function which displays the text inside it to the standard output (screen).
scanf	Input function - value entered by the user will be assigned to the variable whose address is mentioned.
Control Structures	*if, else if, else* conditional statements and *for* loop are all used in accordance with the C language syntax.

Evaluation Criteria	**Observation**	**Record**
Ability for problem definition and realisation	…../**10**	…../**10**
Ability to design and analyse	…../**10**	…../**10**
Ability to implement and validate	…../**10**	…../**10**

RESULT

The various programs discussing the fundamentals of C programming were successfully explored and understood.

Exp. No: 2
Date: 14/09/23

FUNDAMENTALS OF C PROGRAMMING - II

AIM

To practice the fundamental concepts of C programming.

1) Let *i*, *j*, *s* be integer variables. What is the value of *s* printed after the following outer loop?

CODE

```
#include <stdio.h>
int main() {
	int s, i, j;
	s=i=0;
	while (i<10) {
		j=0;
		while (j<5) {
			if (i!=j)
				++s;
			++j;
		}
		++i;
	}
	printf("%d\n",s);
}
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03_mcqs$ cc q16.c
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03_mcqs$ ./a.out
45
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03_mcqs$
```

The value of *s*, printed after the given outer loop, is 45.

2) Consider the following program. Suppose the user inputs the values 5, 3, 5, 9 for *a*, *b*, *c* and *d* respectively. What value for *t* does the program print? Describe in an English sentence the value, in terms of the input integers *a*, *b*, *c*, and *d* printed by the above program.

CODE

```
#include <stdio.h>
int main() {
	int a,b,c,d;
	int r,s,t;
	printf("Enter values for a, b, c, d: ");
	scanf("%d %d %d %d", &a, &b, &c, &d);
	if (a>b) r=a;
	else r=b;

	if (c>d) s=c;
	else s=d;

	if (r>s) t=r;
	else t=s;

	printf("\n%d\n",t);

	return 0;
}
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03_mcqs$ cc q17.c
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03_mcqs$ ./a.out
Enter values for a, b, c, d: 5 3 5 9

9
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03_mcqs$
```

This program takes 4 numbers as input. The first two numbers are compared, and the larger of them is stored in *r*. Then, the second two numbers are compared, and the larger of these is stored in *s*; Finally, *r* and *s* are compared, and the larger one is stored in *t*.

In the case of the inputs 5, 3, 5, 9, the value of *t* will be 9, which is what will be printed.

So, essentially, *t* stores the largest of the four numbers entered by the user.

3) How many times is the statement *i*=i+1*; in the following for loop executed? How many times is the loop condition *i<100* checked in the loop? What is the value stored in the variable *i* immediately after the loop terminates?

CODE

```
#include <stdio.h>
int main() {
	int i;
	for (i=1;i<100;++i) {
		printf("%d\n",i);
		i*=i+1;
	}
	return 0;
}
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03_mcqs$ cc q18.c
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03_mcqs$ ./a.out
1
3
13

Final Value of i = 183
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03_mcqs$
```

The initial value of *i* is 1. The condition *i<100* is checked, found to be true, and so we enter the loop, where the value of *i* is printed. Then, *i* is incremented inside the loop by multiplying it with *i+1*. So, *i* becomes 2. It is then, again, incremented by 1 by the *for* loop.

So, in the second iteration, *i* holds the value of 3. Again, the condition *i<100* is checked, which evaluates to true. So 3 is printed, and two increments follow. First, *i*=3 is multiplied by *i+1*, which is 4. So *i* becomes 12. It is then incremented by 1, so the value of *i* is 13.

Now, we go to the third iteration. The condition *i<100* is again true, so 13 is printed. *i*=i+1* is now *i*=13*14, which is 182, and *i*++ is 183.

Again, we check if *i<100*. This time, the condition is NOT true, so the loop terminates.

***i*=i+1* is executed 3 times.**
***i<100* is checked 4 times.**
***i* has the value 183 after the loop terminates.**

4) The *printf()* function returns the number of characters it prints on *stdout* (screen). What value will be stored in *count* after the execution of the following code?

CODE

```
#include <stdio.h>
int main() {
	int count, n=100;
	count=printf("\nn:%d\n",n);
	printf("Count=%d\n",count);
	return 0;
}
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03_mcqs$ cc q19.c
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03_mcqs$ ./a.out

n:100
Count=7
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03_mcqs$
```

The value of *count* after the execution of the above code is 7.

The escape sequence *\n* is also counted as a character. So the two *\n*, along with the remaining 5 characters, add up to 7, which is what is returned by the *printf()* function and stored in the variable *count*.

5) What will be the contents of the variables *a* and *b* after the execution of the following code on the input *I do not know*?

CODE

```
#include <stdio.h>
int main() {
	char a,b;
	scanf("%c do not %c",&a,&b);
	printf("a=%c, and b=%c\n",a,b);
	return 0;
}
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03_mcqs$ cc q20.c
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03_mcqs$ ./a.out
I do not know
a=I, and b=k
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03_mcqs$
```

***a* has the value 'I', and *b* has the value 'k'.**

The variables *a* and *b* are *char* variables, not *string* variables. So only one character can be stored in these variables. This is why, despite the user giving the value 'know' to *b*, it only takes the value 'k'.

6) Write an iterative function that reads positive integers $a_0, a_1, \ldots, a_{n-1}$ (not necessarily in that order). The function computes and prints the value of $h(a_0, a_1, \ldots, a_{n-1})$ as a rational number in the form p/q and also its floating-point value. The number of terms, that is, *n*, is supplied to the function as an argument. Make clear in your code in which order you are reading and processing $a_0, a_1, \ldots, a_{n-1}$. There is no need to use an array.

```
void cfracitr ( int n )
{
int num, den; /* Numerator and denominator */
/* Declare other int variables, if necessary */

...

printf("Value = %d/%d = %lf\n", num, den, (double)num / (double)den);
}
```

CODE

```
#include <stdio.h>
void cfracitr(int n) {
   int num, den, i, temp_num, temp_den, gcd, smaller, j;

   num=1;
   den=0;

   printf("Enter %d positive integers:\n", n);

   for (i=0;i<n;i++) {
      int ai;
      printf("Enter a%d: ", i);
      scanf("%d",&ai);

      temp_num=num*ai+den;
      temp_den=num;

      num=temp_num;
      den=temp_den;

      gcd=1;
      smaller=(num<den) ? num:den;
      for (j=2;j<=smaller;j++) {
         if (num%j==0 && den%j==0) {
            gcd = j;
         }
      }
```

```
        num/=gcd;
        den/=gcd;

        printf("Value = %d/%d = %.3f\n", num, den, (float)num/den);
    }
}

int main() {
    int n;
    printf("Enter the number of terms (n): ");
    scanf("%d", &n);

    if (n>0)
        cfracitr(n);
    else
        printf("Invalid input.\n");

    return 0;
}
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03_mcqs$ cc q21.c
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03_mcqs$ ./a.out
Enter the number of terms (n): 4
Enter 4 positive integers:
Enter a0: 3
Value = 3/1 = 3.000
Enter a1: 4
Value = 13/3 = 4.333
Enter a2: 5
Value = 68/13 = 5.231
Enter a3: 6
Value = 421/68 = 6.191
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03_mcqs$
```

7) Let $a_1, a_2, \ldots, a_n$ be a sequence of positive integers. An increasing subsequence of length l is a contiguous block $a_i, a_{i+1}, \ldots, a_{i+l-1}$ satisfying $a_i < a_{i+1} < \ldots < a_{i+l-1}$. Write a C program to read a sequence of positive integers and to print the length of the longest increasing subsequence in it. In order to terminate the sequence, the user should enter zero or a negative value. Your program must contain only one loop. Both scanning the next integer and processing the scanned integer should be done in that loop. Write no functions other than *main()*. Do not use any array.

CODE

```
#include <stdio.h>
int main() {
	int current, previous, temp=0, lis=0;
	printf("Enter an integer: ");
	scanf("%d",&current);
	previous=current;
	while (current>=0) {
		temp++;
		printf("Enter an integer: ");
		scanf("%d",&current);
		if (current<=0) {
			if (temp>lis)
				lis=temp;
			break;
		}
		if (current<previous) {
			if (temp>lis)
				lis=temp;
			temp=0;
		}
		previous=current;
	}
	printf("\nLength of the Longest Increasing Subsequence = %d\n",lis);
	return 0;
}
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03_mcqs$ cc q22.c
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03_mcqs$ ./a.out
Enter an integer: 3
Enter an integer: 7
Enter an integer: 2
Enter an integer: -1

Length of the Longest Increasing Subsequence = 2
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03_mcqs$
```

8) Explain the output of the following code.

CODE

```
#include <stdio.h>
int main() {
	char x='w';
	switch (x){
		case 'a':
			printf("Append");
			break;
		case 'w':
			printf("Write");
		case 'r':
			printf("Read");
			break;
		default:
			printf("Open");
			break;
	}
	printf("\n");
	return 0;
}
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03_mcqs$ cc q23.c
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03_mcqs$ ./a.out
WriteRead
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03_mcqs$ |
```

The obvious assumption would be that only *Write* will be printed, as that is what the *switch* condition matches to. However, notice that there is no *break* statement after that. The *switch* statement executes every statement after a matched *case* until it meets with a *break* statement. This is why both *Write* and *Read* are printed, as the first time a *break* is met with after a matched *case* is after the *printf("Read");* statement.

9) Write a program to print to print the sum of the series $1 + x^2/2! + x^4/4! + \ldots + x^{2N}/2N!$.

CODE

```
#include <stdio.h>
float factorial(int n) {
	if (n==0)
		return 1;
	return n*factorial(n-1);
}
float power(int n, int p) {
	if (p==0)
		return 1;
	return n*power(n,p-1);
}
int main() {
	int i, N;
	float x, S, term;
	S=1.0;
	term=1.0;
	printf("Enter N and x: ");
	scanf("%d %f",&N,&x);
	for (i=2;i<2*N;i+=2) {
		term = power(x,i)/factorial(i);
		S+=term;
	}
	printf("\nThe sum of the series is %.6f.\n",S);
	return 0;
}
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03_mcqs$ cc q24.c
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03_mcqs$ ./a.out
Enter N and x: 4 5

The sum of the series is 61.243053.
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03_mcqs$
```

10) Write a program to print the most significant digit of an integer *n*.

CODE

```
#include <stdio.h>
int main() {
	int n;
	printf("Enter n: ");
	scanf("%d",&n);
	while (n>=10)
		n/=10;
	printf("The most significant digit is %d.\n",n);
	return 0;
}
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03_mcqs$ cc q25.c
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03_mcqs$ ./a.out
Enter n: 7321
The most significant digit is 7.
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03_mcqs$
```

11) Write a C program which reads a sequence of positive integers till the user types −1. It counts the lengths of the non-decreasing sub sequences, and prints the maximum among them. For example, for input {6, 7, 2, 29, 17, 5, 5, 11, 6, 7, 8, −1} the non-decreasing sub sequences are: {6, 7}, {2, 29}, {17}, {5, 5, 11} and {6, 7, 8}. Thus the answer should be 3. Assume that the first integer read is not −1 and a single integer is a sequence of length 1.

CODE

```
#include <stdio.h>
int main() {
	int current, previous, temp=0, lnds=0;
	printf("Enter an integer: ");
	scanf("%d",&current);
	previous=current;
	while (current>=0) {
		temp++;
		printf("Enter an integer: ");
		scanf("%d",&current);
		if (current<0)
			break;
		if (current<previous) {
			if (temp>lnds)
				lnds=temp;
			temp=0;
		}
		previous=current;
	}
	printf("\nLength of the Longest Non-Decreasing Subsequence = %d\n",lnds);
	return 0;
}
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03_mcqs$ cc q26.c
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03_mcqs$ ./a.out
Enter an integer: 1
Enter an integer: 2
Enter an integer: 3
Enter an integer: 0
Enter an integer: 2
Enter an integer: -1

Length of the Longest Non-Decreasing Subsequence = 3
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03_mcqs$
```

12) How many *s are printed by the following code segment?

CODE

```
#include <stdio.h>
int main() {
	int i,j;
	for (i=0;i<10;i++) {
		for (j=i;j>0;j--)
			printf("*");
	}
	printf("\n");
	return 0;
}
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03_mcqs$ cc q27.c
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03_mcqs$ ./a.out
*********************************************
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03_mcqs$ ./a.out | wc -m
46
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03_mcqs$
```

The above code prints *s equal to the sum of the first *n-1* natural numbers, where *n* is the upper limit of the outer loop. In this case, the upper limit is 10, so * equal to the sum of the first 9 natural numbers is printed.

1+2+3+4+5+6+7+8+9=45.

This is corroborated by the *wc -m* command, which here counts the number of characters of the output. It includes the *\n* as a character, so excluding that, we do get 45, the expected answer.

13) Fill in the blanks (for appropriate actions mentioned in brackets):

_____

/* Include a header file for I/O operation. (1) */
main()
{
float x,y;
int z;
printf("Read x and y \n");
scanf("_____",__,__); /* Read x and y in appropriate format. (1)*/ z= _____(int)____(x+y); /* Type-cast. (1) */
if (z ____ 0) /* Check equality with zero. (1) */
printf("x=_ __, y=__ ___\n",____, ___); /* Print values of x, and y. (1) */

CODE

```
#include <stdio.h>
int main() {
    float x,y;
    int z;
    printf("Read x and y\n");
    scanf("%f %f",&x,&y);
    z=(int)(x+y);
    if (z==0)
        printf("x=%f, y=%f\n",x,y);
    return 0;
}
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03_mcqs$ cc q28.c
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03_mcqs$ ./a.out
Read x and y
5 10
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03_mcqs$ ./a.out
Read x and y
0.2 0.79
x=0.200000, y=0.790000
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03_mcqs$
```

14) Write a C-program to print the first ten multiples of 5 using a while loop.

[Sample Output: 5 10 15 20 25 30 35 40 45 50]

CODE

```
#include <stdio.h>
int main() {
	int i=5;
	while (i<=50) {
		printf("%d\t",i);
		i+=5;
	}
	printf("\n");
	return 0;
}
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03_mcqs$ cc q29.c
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03_mcqs$ ./a.out
5	10	15	20	25	30	35	40	45	50
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03_mcqs$
```

15) Consider the following code:

CODE

```
#include <stdio.h>
int main() {
	int a=0, b=100;
	if (a) {
		if (b)
			printf("abc\n");
		else
			printf("bcd\n");
	}
	return 0;
}
```

(a) What will be the output generated by the above code? Explain your reasoning.

Here, *a* is 0. 0 is equivalent to false, and so the compiler doesn't go into the *if* block, meaning nothing will be printed to the standard output.

OUTPUT (a)

```
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03_mcqs$ cc q30.c
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03_mcqs$ ./a.out
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03_mcqs$
```

(b) What will be the output generated by the above code when a = -100 and b = 100? Explain your reasoning.

Any value other than 0 is equivalent to true. So when *a* is -100, the compiler goes into the *if* block, where the next condition is checked. Since *b* is also a non-zero value, the statement under the nested *if* is executed. Therefore, the output will be *abc*.

OUTPUT (b)

```
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03_mcqs$ cc q30.c
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03_mcqs$ ./a.out
abc
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03_mcqs$
```

16) Read the symbol of a binary arithmetic operator (such as +, -, *, /) and its two operands from the keyboard and perform the operation on those two operands depending upon the operator entered by the user. Print the result accordingly.

CODE

```
#include <stdio.h>
int main() {
	char symbol;
	int a, b, result;
	printf("Enter the Operation (+ or - or * or /): ");
	scanf("%c", &symbol);
	if (symbol=='+' || symbol=='-' || symbol=='*' || symbol=='/') {
		printf("Enter Operand 1: ");
		scanf("%d", &a);
		printf("Enter Operand 2: ");
		scanf("%d", &b);
		if (symbol=='+') result=a+b;
		else if (symbol=='-') result=a-b;
		else if (symbol=='*') result=a*b;
		else result=a/b;
		printf("\n%d %c %d = %d\n",a,symbol,b,result);
	} else
		printf("Invalid Operator.\n");
	return 0;
}
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03$ cc read_symbol_operation.c
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03$ ./a.out
Enter the Operation (+ or - or * or /): +
Enter Operand 1: 3
Enter Operand 2: 7

3 + 7 = 10
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03$ ./a.out
Enter the Operation (+ or - or * or /): -
Enter Operand 1: 3
Enter Operand 2: 7

3 - 7 = -4
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03$ ./a.out
Enter the Operation (+ or - or * or /): *
Enter Operand 1: 3
Enter Operand 2: 7

3 * 7 = 21
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03$ ./a.out
Enter the Operation (+ or - or * or /): /
Enter Operand 1: 21
Enter Operand 2: 7

21 / 7 = 3
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03$ ./a.out
Enter the Operation (+ or - or * or /): x
Invalid Operator.
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03$
```

17) Write a C program to do the following:

- Read the marks (maximum 100) obtained by a student in a subject.
- Convert the mark entered to an appropriate grade as per the following scheme.

(i)	Grade = X	if mark is greater than or equal to 90
(ii)	Grade = A	if mark is between 80 and 90
(iii)	Grade = B	if mark is between 70 and 80
(iv)	Grade = C	if mark is between 60 and 70
(v)	Grade = D	if mark is between 50 and 60
(vi)	Grade = P	if mark is between 35 and 50
(vii)	Grade = F	otherwise

- Print the grade.

CODE

```
#include <stdio.h>
int main() {
	int mark;
	char grade;
	printf("Enter your marks: ");
	scanf("%d",&mark);
	if (mark>=0 && mark<=100) {
		if (mark>=90)
			grade='X';
		else if (mark>=80)
			grade='A';
		else if (mark>=70)
			grade='B';
		else if (mark>=60)
			grade='C';
		else if (mark>=50)
			grade='D';
		else if (mark>=35)
			grade='P';
		else
			grade='F';
		printf("Your grade is %c.\n",grade);
	} else
		printf("Invalid Marks.\n");
	return 0;
}
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03$ cc grade_calculation.c
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03$ ./a.out
Enter your marks: 99
Your grade is X.
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03$ ./a.out
Enter your marks: 87
Your grade is A.
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03$ ./a.out
Enter your marks: 73
Your grade is B.
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03$ ./a.out
Enter your marks: 66
Your grade is C.
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03$ ./a.out
Enter your marks: 51
Your grade is D.
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03$ ./a.out
Enter your marks: 36
Your grade is P.
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03$ ./a.out
Enter your marks: 34
Your grade is F.
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03$ ./a.out
Enter your marks: 101
Invalid Marks.
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03$ ./a.out
Enter your marks: -2
Invalid Marks.
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03$
```

18) A point in a plane can be represented by its X and Y co-ordinates. Write C programs to do the following:

(i) Read the coordinates of a pair of points and find the distance between these two points and print the result.

CODE

```
#include <stdio.h>
#include <math.h>

int main() {
	float x1, y1, x2, y2;
	printf("X1: ");
	scanf("%f",&x1);
	printf("Y1: ");
	scanf("%f",&y1);
	printf("X2: ");
	scanf("%f",&x2);
	printf("Y2: ");
	scanf("%f",&y2);

	float distance;
	distance=sqrt(pow(x2-x1,2)+pow(y2-y1,2));
	printf("\nPoint A: (%f,%f)\n",x1,y1);
	printf("Point B: (%f,%f)\n",x2,y2);
	printf("Distance = %f units\n",distance);
	return 0;
}
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03$ cc xy_distance.c -lm
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03$ ./a.out
X1: 1
Y1: 3
X2: 1
Y2: 7

Point A: (1.000000,3.000000)
Point B: (1.000000,7.000000)
Distance = 4.000000 units
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03$
```

(ii) Read the coordinates of three points and then check whether they can constitute a triangle. If so, find the area and perimeter of the triangle and check whether it is a scalene, isosceles or an equilateral triangle.

CODE

```
#include <stdio.h>
#include <math.h>

float perimeter(float a, float b, float c) {
	return a+b+c;
}

float area(float a, float b, float c) {
	float s = perimeter(a,b,c)/2;
	return sqrt(s*(s-a)*(s-b)*(s-c));
}

int main() {
	float x1, y1, x2, y2, x3, y3;
	printf("X1: ");
	scanf("%f",&x1);
	printf("Y1: ");
	scanf("%f",&y1);
	printf("X2: ");
	scanf("%f",&x2);
	printf("Y2: ");
	scanf("%f",&y2);
	printf("X3: ");
	scanf("%f",&x3);
	printf("Y3: ");
	scanf("%f",&y3);

	float a,b,c;

	a=sqrt(pow(x2-x1,2)+pow(y2-y1,2));
	b=sqrt(pow(x3-x2,2)+pow(y3-y2,2));
	c=sqrt(pow(x1-x3,2)+pow(y1-y3,2));

	if (a+b>c && b+c>a && c+a>b) {
		printf("\nThese points will form a triangle.\n");
		printf("Perimeter = %f units\n",perimeter(a,b,c));
		printf("Area = %f sq. units\n",area(a,b,c));

		if (a==b && b==c && c==a)
```

```
            printf("The triangle is equilateral.\n");
        else if (a!=b && b!=c && c!=a)
            printf("The triangle is scalene.\n");
        else
            printf("The triangle is isosceles.\n");


    } else {
        printf("These points do not form a triangle.\n");
    }
    return 0;
}
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03$ cc xyz_triangle.c -lm
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03$ ./a.out
X1: 0
Y1: 0
X2: 0
Y2: 1
X3: 1
Y3: 0

These points will form a triangle.
Perimeter = 3.414214 units
Area = 0.500000 sq. units
The triangle is isosceles.
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03$ ./a.out
X1: 0
Y1: 0
X2: 7
Y2: 0
X3: 0
Y3: 3

These points will form a triangle.
Perimeter = 17.615772 units
Area = 10.499996 sq. units
The triangle is scalene.
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03$ ./a.out
X1: 1
Y1: 1
X2: 1
Y2: 1
X3: 2
Y3: 2

These points do not form a triangle.
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03$
```

19) Read a sequence of integer numbers (terminated by 0) and find the maximum, minimum and average value of the numbers (excluding 0) that have been entered.

CODE

```
#include <stdio.h>
int main() {
	int no=0, e;
	printf("Enter Number %d: ",no+1);
	scanf("%d",&e);
	no++;
	int max=e, min=e, sum=e;
	while (e!=0) {
		printf("Enter Number %d: ",no+1);
		scanf("%d",&e);
		if (e==0)
			break;
		no++;
		if (max < e)
			max=e;
		if (min > e)
			min=e;
		sum+=e;
	}
	float avg=sum/no;
	printf("\nMaximum Value = %d",max);
	printf("\nMinimum Value = %d",min);
	printf("\nAverage = %f\n",avg);
	return 0;
}
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03$ cc read_till_0.c
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03$ ./a.out
Enter Number 1: 37
Enter Number 2: 73
Enter Number 3: 21
Enter Number 4: -30
Enter Number 5: 0

Maximum Value = 73
Minimum Value = -30
Average = 25.000000
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03$
```

20) Read a sequence of characters (terminated by new line character *n*) and count the number of vowels entered.

CODE

```
#include <stdio.h>
int main() {
	char c;
	int vowel_count=0;
	printf("Provide a number of characters (press enter to stop): ");
	scanf("%c",&c);
	while (c!='\n') {
		if (c=='a' || c=='e' || c=='i' || c=='o' || c=='u' || c=='A' || c=='E' || c=='I' || c=='O' || c=='U')
			vowel_count++;
		scanf("%c",&c);
	}
	printf("No. of Vowels: %d\n",vowel_count);
	return 0;
}
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03$ cc read_till_n.c
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03$ ./a.out
Provide a number of characters (press enter to stop): Yuvarajistall
No. of Vowels: 5
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03$
```

21) A number is called a perfect number, if the number is equal to the sum of all its positive divisors except the number itself. For example, (6=1+2+3, 28=1+2+4+7+14). Find and print all the perfect numbers less than or equal to 1000.

CODE

```
#include <stdio.h>
int factor_sum(int n) {
	int i,s=0;
	for(i=1;i<=n/2;i++) {
		if (n%i==0)
			s+=i;
	}
	return s;
}
int main() {
	int i;
	printf("Perfect Numbers upto 1000\n");
	for (i=1;i<=1000;i++) {
		if (factor_sum(i)==i)
			printf("%d ",i);
	}
	printf("\n");
	return 0;
}
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03$ cc perfect_numbers.c
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03$ ./a.out
Perfect Numbers upto 1000
6 28 496
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03$
```

22) Read an integer number and convert it to its hexadecimal equivalent representation.

CODE

```
#include <stdio.h>
#include <string.h>
void rev(char str[]) {
   int length = strlen(str);
   int i, j;
   char temp;
   for (i = 0, j = length - 1; i < j; i++, j--) {
      temp = str[i];
      str[i] = str[j];
      str[j] = temp;
   }
}
int main() {
      int n;
      printf("Enter an Integer: ");
      scanf("%d",&n);
      int x=n,r;
      char c;
      char hexadecimal[100]="";
      while (x!=0) {
            r=x%16;
            if (r<10)
                  c=r+'0';
            else
                  c=r-10+'A';
            char temp[2] = {c,'\0'};
            strcat(hexadecimal, temp);
            x/=16;
      }
      rev(hexadecimal);
      printf("\n%d in Hexadecimal = %s\n", n, hexadecimal);
      return 0;
}
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03$ cc int_to_hex.c
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03$ ./a.out
Enter an Integer: 60
60 in Hexadecimal = 3C
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03$
```

23) Write a program that reads three integers representing time of a day. The integers are SS, MM, HH, representing seconds, minutes and hours. Write a program to print “valid time” if the integers represent a valid time of the day.

CODE

```
#include <stdio.h>
int main() {
	int s, m, h;
	printf("Time of the Day\n\n");
	printf("Hours: ");
	scanf("%d",&h);
	printf("Minutes: ");
	scanf("%d",&m);
	printf("Seconds: ");
	scanf("%d",&s);

	if (h>=0 && h<24 && m>=0 && m<60 && s>=0 && s<60) {
		printf("\n%d:%d:%d",h,m,s);
		printf("\nValid Time.\n");
	}
	else
		printf("\nInvalid Time.\n");
	return 0;
}
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03$ cc valid_time.c
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03$ ./a.out
Time of the Day

Hours: 9
Minutes: 55
Seconds: 28

9:55:28
Valid Time.
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03$ ./a.out
Time of the Day

Hours: 11
Minutes: 60
Seconds: 3

Invalid Time.
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03$
```

24) Write a program to read three integers and print the largest among them.

CODE

```
#include <stdio.h>
int main() {
	int a,b,c;
	printf("Number 1: ");
	scanf("%d",&a);
	printf("Number 2: ");
	scanf("%d",&b);
	printf("Number 3: ");
	scanf("%d",&c);

	if (a==b && b==c)
		printf("\nYou've entered the same number.\n");
	else if (a>b && a>c)
		printf("\n%d is the largest.\n",a);
	else if (b>c)
		printf("\n%d is the largest.\n",b);
	else
		printf("\n%d is the largest.\n",c);
	return 0;
}
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03$ cc largest_among_3.c
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03$ ./a.out
Number 1: 3
Number 2: 21
Number 3: 7

21 is the largest.
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03$
```

25) Write a program to compute and print the taxi fare based on the following chart. Total number of kilometres travelled will be input by the user as a floating point number.

First 12 KM:	Rs. 100/-
Next 4 KM:	Rs. 8 / KM
Next 4 KM:	Rs 6 / KM
Above 20 KM:	Rs 5 / KM

CODE

```
#include <stdio.h>
int main() {
	float km;
	printf("Total Distance Travelled (in km) = ");
	scanf("%f",&km);
	if (km<0)
		printf("\nInvalid.\n");
	else if (km==0)
		printf("\nYou have not travelled, and so you don't need to make any payment.\n");
	else {
		int price, next;
		if (km<=12)
			price=100;
		else if (km<=16) {
			next=8*(km-12);
			price=100+next;
		}
		else if (km<=20) {
			next=6*(km-16);
			price=132+next;
		}
		else {
			next=5*(km-20);
			price=156+next;
		}
		printf("\nTotal Fare = Rs. %d\n",price);
	}
	return 0;
}
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03$ cc taxi_fare.c
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03$ ./a.out
Total Distance Travelled (in km) = -3

Invalid.
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03$ ./a.out
Total Distance Travelled (in km) = 0

You have not travelled, and so you don't need to make any payment.
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03$ ./a.out
Total Distance Travelled (in km) = 7

Total Fare = Rs. 100
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03$ ./a.out
Total Distance Travelled (in km) = 15

Total Fare = Rs. 124
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03$ ./a.out
Total Distance Travelled (in km) = 17

Total Fare = Rs. 138
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03$ ./a.out
Total Distance Travelled (in km) = 21

Total Fare = Rs. 161
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03$
```

26) Read n integers and print the third largest among them. *n* is input by user. Don't use any array.

CODE

```
#include <stdio.h>
int main() {
	int n, a, b, c, max, smax, tmax;
	printf("Total Number of Integers (3 or more): ");
	scanf("%d",&n);
	if (n<3)
		printf("\nAt least 3 integers are required to compute the third largest.\n");
	else {
		printf("\nValue 1: ");
		scanf("%d",&a);
		printf("Value 2: ");
		scanf("%d",&b);
		printf("Value 3: ");
		scanf("%d",&c);

		if (a>b && a>c) {
			max=a;
			if (b>c) {
				smax=b;
				tmax=c;
			} else {
				smax=c;
				tmax=b;
			}
		} else if (b>a && b>c) {
			max=b;
			if (a>c) {
				smax=a;
				tmax=c;
			} else {
				smax=c;
				tmax=a;
			}
		} else {
			max=c;
			if (b>a) {
				smax=b;
				tmax=a;
			} else {
				smax=a;
```

```
                tmax=b;
            }
        }
        int i, temp;
        for (i=4;i<=n;i++) {
            printf("Value %d: ",i);
            scanf("%d",&temp);
            if (temp>max) {
                tmax=smax;
                smax=max;
                max=temp;
            } else if (temp>smax) {
                tmax=smax;
                smax=temp;
            } else if (temp>tmax) {
                tmax=temp;
            }
        }
        printf("\nLargest = %d",max);
        printf("\nSecond Largest = %d",smax);
        printf("\nThird Largest = %d\n",tmax);
    }
    return 0;
}
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03$ cc third_largest_among_n.c
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03$ ./a.out
Total Number of Integers (3 or more): 2

At least 3 integers are required to compute the third largest.
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03$ ./a.out
Total Number of Integers (3 or more): 7

Value 1: 21
Value 2: 3333
Value 3: 7
Value 4: 732130
Value 5: 37
Value 6: 103
Value 7: 127

Largest = 732130
Second Largest = 3333
Third Largest = 127
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03$
```

27) Read any number and print the sum of its digits.

CODE

```
#include <stdio.h>
int calculate_sum(int n) {
	if (n<10)
		return n;
	return (n%10)+calculate_sum(n/10);
}

int main() {
	int n;
	printf("Enter a Number: ");
	scanf("%d",&n);
	printf("Sum of Digits = %d\n",calculate_sum(n));
	return 0;
}
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03$ cc sum_of_digits.c
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03$ ./a.out
Enter a Number: 3721
Sum of Digits = 13
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03$ ./a.out
Enter a Number: 1
Sum of Digits = 1
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03$ ./a.out
Enter a Number: 0
Sum of Digits = 0
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03$ ./a.out
Enter a Number: 100
Sum of Digits = 1
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03$
```

28) Find the sum of the first *n* terms of the following series.

$$1 + x + x^2/2! + x^3/3! + \ldots$$

Values of x and n are input by the user. Do not use the factorial or power functions available in *math.h*.

CODE

```
#include <stdio.h>
float factorial(int n) {
	if (n==0)
		return 1;
	return n*factorial(n-1);
}
float power(int n, int pow) {
	if (pow==0)
		return 1;
	return n*power(n,pow-1);
}
int main() {
	int x, n, i;
	float s=0;
	printf("x = ");
	scanf("%d",&x);
	printf("n = ");
	scanf("%d",&n);
	for (i=0;i<n;i++)
		s+=(power(x,i)/factorial(i));
	printf("Sum of %d Terms of Series with x as %d = %.6f\n",n,x,s);
	return 0;
}
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03$ cc sum_of_series.c
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03$ ./a.out
x = 3
n = 7
Sum of 7 Terms of Series with x as 3 = 19.412500
warspinix@Yuvaraj-Laptop:~/c_programs/exp_03$
```

Evaluation Criteria	Observation	Record
Ability for problem definition and realisation	…../**10**	…../**10**
Ability to design and analyse	…../**10**	…../**10**
Ability to implement and validate	…../**10**	…../**10**

RESULT

More fundamental programs were worked out in C, and the desired outputs were obtained.

Exp. No: 3
Date: 21/09/23

IMPLEMENTATION OF SINGLE DIMENSIONAL, MULTI-DIMENSIONAL ARRAYS AND FUNCTIONS

AIM

To explore single dimensional arrays, multi-dimensional arrays and functions.

SUM OF NUMBERS IN A 1 DIMENSIONAL ARRAY

```
#include <stdio.h>

int calculate_sum(int arr[],int length) {
	if (length==1)
		return arr[0];
	return arr[0]+calculate_sum(&arr[1],length-1);
}

int main() {
	int n, i, e;
	printf("Number of Elements in Array: ");
	scanf("%d",&n) ;
	if (n>0) {
		int arr[n];
		for (i=0;i<n;i++) {
			printf("Element %d: ",i+1);
			scanf("%d",&e);
			arr[i]=e;
		}
		int s = calculate_sum(arr,n);
		printf("\n\nSum of Elements = %d\n",s);
	} else if (n==0) {
		printf("\n\nSum of Elements = 0\n");
	} else {
		printf("\n\nPlease enter a non-negative integer.\n");
	}

	return 0;
}
```

OUTPUT

There are mainly 3 conditions to consider.

The first is that the user enters a positive integer for *n*, which stores the number of elements in the array, or simply, the length of the array. When a positive integer is provided for *n*, we can go ahead and get the individual array elements from the user, and then calculate the sum of the array.

If the user enters 0, then no elements will be received. The array will be empty and the sum of the elements in it will be 0.

If the user enters a negative integer, we provide a prompt that informs them such values aren't valid/allowed. The length of an array should always be positive.

```
warspinix@Yuvaraj-Laptop:~/c_programs/exp_02$ cc sum_1D_array.c
warspinix@Yuvaraj-Laptop:~/c_programs/exp_02$ ./a.out
Number of Elements in Array: 7

Element 1: 33
Element 2: 37
Element 3: 73
Element 4: 21
Element 5: 23
Element 6: 77
Element 7: 3

Sum of Elements = 267
warspinix@Yuvaraj-Laptop:~/c_programs/exp_02$ ./a.out
Number of Elements in Array: 0

Sum of Elements = 0
warspinix@Yuvaraj-Laptop:~/c_programs/exp_02$ ./a.out
Number of Elements in Array: -5

Please enter a non-negative integer.
warspinix@Yuvaraj-Laptop:~/c_programs/exp_02$
```

SUM OF NUMBERS IN A 2 DIMENSIONAL ARRAY

```
#include <stdio.h>
int main() {
	int rows, columns, i, j, e;
	printf("No. of Rows: ");
	scanf("%d",&rows);
	printf("No. of Columns: ");
	scanf("%d",&columns);
	int arr[rows][columns];
	for (i=0;i<rows;i++) {
		printf("\nRow %d Elements\n",i+1);
		for (j=0;j<columns;j++) {
			printf("Element %d: ",j+1);
			scanf("%d",&e);
			arr[i][j]=e;
		}
	}
	printf("\n");
	int sum=0;
	int temp_sum;
	for (i=0;i<rows;i++) {
		temp_sum=0;
		for (j=0;j<columns;j++)
			temp_sum += arr[i][j];
		sum += temp_sum;
		printf("Sum of Row %d Elements = %d\n",i+1,temp_sum);
	}
	printf("\nTotal Array Sum = %d\n",sum);
	return 0;
}
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/c_programs/exp_02$ cc sum_2D_array.c
warspinix@Yuvaraj-Laptop:~/c_programs/exp_02$ ./a.out
No. of Rows: 2
No. of Columns: 3

Row 1 Elements
Element 1: 1
Element 2: 2
Element 3: 3

Row 2 Elements
Element 1: 33
Element 2: 77
Element 3: 73

Sum of Row 1 Elements = 6
Sum of Row 2 Elements = 183

Total Array Sum = 189
warspinix@Yuvaraj-Laptop:~/c_programs/exp_02$
```

COMBINATIONS WITH FUNCTION

```
#include <stdio.h>
int factorial(int);
int combination(int, int);
int main() {
	int n, r;
	printf("Total Number of Unordered Items: ");
	scanf("%d",&n);
	printf("No. of Selections: ");
	scanf("%d",&r);

	if (n>r && r>0) {
		int RESULT = combination(n,r);
		printf("\nTotal No. of Combinations = %d\n",RESULT);
	} else {
		printf("\nThe condition n>r>0 is not satisfied.\n");
	}
	return 0;
}
int factorial(int n) {
	if (n==0)
		return 1;
	return n*factorial(n-1);
}
int combination(int n, int r) {
	int fn = factorial(n);
	int fr = factorial(r);
	int fn_r = factorial(n-r);
	return fn/(fr*fn_r);
}
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/c_programs/exp_02$ cc ncr_function.c
warspinix@Yuvaraj-Laptop:~/c_programs/exp_02$ ./a.out
Total Number of Unordered Items: 8
No. of Selections: 3

Total No. of Combinations = 56
warspinix@Yuvaraj-Laptop:~/c_programs/exp_02$
```

COMBINATIONS USING HEADER FILE

HEADER FILE - *combination.h*

```
int factorial(int n) {
	if (n==0)
		return 1;
	return n*factorial(n-1);
}
int combination(int n, int r) {
	int fn = factorial(n);
	int fr = factorial(r);
	int fn_r = factorial(n-r);
	return fn/(fr*fn_r);
}
```

PROGRAM FILE - *ncr_function.c*

```
#include <stdio.h>
#include "combination.h"
int main() {
	int n, r;
	printf("Total Number of Unordered Items: ");
	scanf("%d",&n);
	printf("No. of Selections: ");
	scanf("%d",&r);
	if (n>r && r>0) {
		int RESULT = combination(n,r);
		printf("\nTotal No. of Combinations = %d\n",RESULT);
	} else
		printf("\nThe condition n>r>0 is not satisfied.\n");
	return 0;
}
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/c_programs/exp_02$ cc ncr_header.c
warspinix@Yuvaraj-Laptop:~/c_programs/exp_02$ ./a.out
Total Number of Unordered Items: 7
No. of Selections: 3

Total No. of Combinations = 35
warspinix@Yuvaraj-Laptop:~/c_programs/exp_02$
```

MAXIMUM AND MINIMUM FROM ARRAY OF INTEGERS/FLOAT

```
#include <stdio.h>
int main() {
	int type, n;
	printf("Press 1 for Integer, 0 for Float: ");
	scanf("%d",&type);

	if (type==0 || type==1) {
		printf("\nNo. of Elements in the Array: ");
		scanf("%d",&n);

		if (type==1) {
			int arr[n];
			int e, i;

			printf("\n");

			for (i=0;i<n;i++) {
				printf("Element %d: ",i+1);
				scanf("%d",&e);
				arr[i]=e;
			}

			int min=arr[0];
			int max=arr[0];

			for (i=0;i<n;i++) {
				if (arr[i] > max)
					max = arr[i];
				if (arr[i] < min)
					min = arr[i];
			}

			printf("\nMaximum Element: %d",max);
			printf("\nMinimum Element: %d\n",min);

		} else {
			float arr[n];
			float e;
			int i;

			printf("\n");

			for (i=0;i<n;i++) {
```

```
                printf("Element %d: ",i+1);
                scanf("%f",&e);
                arr[i]=e;
            }



            float min=arr[0];
            float max=arr[0];

            for (i=0;i<n;i++) {
                if (arr[i] > max)
                    max = arr[i];
                if (arr[i] < min)
                    min = arr[i];
            }

            printf("\nMaximum Element: %f",max);
            printf("\nMinimum Element: %f\n",min);

        }

    } else {
        printf("\nInvalid Input.\n");
    }
    return 0;
}
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/c_programs/exp_02$ cc max_min_array.c
warspinix@Yuvaraj-Laptop:~/c_programs/exp_02$ ./a.out
Press 1 for Integer, 0 for Float: 1

No. of Elements in the Array: 5

Element 1: 1
Element 2: 45
Element 3: 3
Element 4: 33
Element 5: 22

Maximum Element: 45
Minimum Element: 1
warspinix@Yuvaraj-Laptop:~/c_programs/exp_02$
```

```
warspinix@Yuvaraj-Laptop:~/c_programs/exp_02$ cc max_min_array.c
warspinix@Yuvaraj-Laptop:~/c_programs/exp_02$ ./a.out
Press 1 for Integer, 0 for Float: 0

No. of Elements in the Array: 7

Element 1: 0.4
Element 2: 3.3
Element 3: -1.54
Element 4: 33
Element 5: -1.541
Element 6: 33.01
Element 7: 33.011

Maximum Element: 33.011002
Minimum Element: -1.541000
warspinix@Yuvaraj-Laptop:~/c_programs/exp_02$
```

```
warspinix@Yuvaraj-Laptop:~/c_programs/exp_02$ cc max_min_array.c
warspinix@Yuvaraj-Laptop:~/c_programs/exp_02$ ./a.out
Press 1 for Integer, 0 for Float: 2

Invalid Input.
warspinix@Yuvaraj-Laptop:~/c_programs/exp_02$
```

MATRIX ADDITION

```
#include <stdio.h>
int main() {
	int rows, columns, i, j, e;
	printf("No. of Rows: ");
	scanf("%d",&rows);
	printf("No. of Columns: ");
	scanf("%d",&columns);

	if (rows>0 && columns>0) {
		int m1[rows][columns];
		printf("\nMatrix A Elements\n");
		for (i=0;i<rows;i++) {
			printf("\nRow %d Elements\n",i+1);
			for (j=0;j<columns;j++) {
				printf("Element %d: ",j+1);
				scanf("%d",&e);
				m1[i][j]=e;
			}
		}

		int m2[rows][columns];
		printf("\nMatrix B Elements\n");
		for (i=0;i<rows;i++) {
			printf("\nRow %d Elements\n",i+1);
			for (j=0;j<columns;j++) {
				printf("Element %d: ",j+1);
				scanf("%d",&e);
				m2[i][j]=e;
			}
		}

		printf("\nMatrix A\n\n");

		for (i=0;i<rows;i++) {
			for (j=0;j<columns;j++) {
				printf("%d  ",m1[i][j]);
			}
			printf("\n");
		}

		printf("\nMatrix B\n\n");
```

```
        for (i=0;i<rows;i++) {
            for (j=0;j<columns;j++) {
                printf("%d  ",m2[i][j]);
            }
            printf("\n");
        }

        int msum[rows][columns];
        printf("\nSum of Matrix A and Matrix B\n\n");

        for (i=0;i<rows;i++) {
            for (j=0;j<columns;j++) {
                msum[i][j] = m1[i][j] + m2[i][j];
                printf("%d  ",msum[i][j]);
            }
            printf("\n");
        }
    } else {
        printf("\n\nRows and Columns should be greater than 0.\n");
    }
    return 0;
}
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/c_programs/exp_02$ cc matrix_addition.c
warspinix@Yuvaraj-Laptop:~/c_programs/exp_02$ ./a.out
No. of Rows: 3
No. of Columns: 4

Matrix A Elements

Row 1 Elements
Element 1: 1
Element 2: 2
Element 3: 3
Element 4: 4

Row 2 Elements
Element 1: 5
Element 2: 6
Element 3: 7
Element 4: 8

Row 3 Elements
Element 1: 9
Element 2: 8
Element 3: 7
Element 4: 6

Matrix B Elements

Row 1 Elements
Element 1: 5
Element 2: 4
Element 3: 3
Element 4: 2

Row 2 Elements
Element 1: 1
Element 2: 2
Element 3: 3
Element 4: 4
```

```
Row 3 Elements
Element 1: 5
Element 2: 6
Element 3: 7
Element 4: 8

Matrix A

1  2  3  4
5  6  7  8
9  8  7  6

Matrix B

5  4  3  2
1  2  3  4
5  6  7  8

Sum of Matrix A and Matrix B

6  6  6  6
6  8  10  12
14  14  14  14
warspinix@Yuvaraj-Laptop:~/c_programs/exp_02$
```

MATRIX MULTIPLICATION

```
#include <stdio.h>
int main() {
	int r1, c1, r2, c2, i, j, k, e;
	printf("Matrix A\n");
	printf("No. of Rows: ");
	scanf("%d",&r1);
	printf("No. of Columns: ");
	scanf("%d",&c1);
	printf("\nMatrix B\n");
	printf("No. of Rows: ");
	scanf("%d",&r2);
	printf("No. of Columns: ");
	scanf("%d",&c2);
	if (r1>0 && c1>0 && r2>0 && c2>0 && c1==r2) {
		int m1[r1][c1];
		printf("\nMatrix A Elements\n");
		for (i=0;i<r1;i++) {
			printf("\nRow %d Elements\n",i+1);
			for (j=0;j<c1;j++) {
				printf("Element %d: ",j+1);
				scanf("%d",&e);
				m1[i][j]=e; } }
		int m2[r2][c2];
		printf("\nMatrix B Elements\n");
		for (i=0;i<r2;i++) {
			printf("\nRow %d Elements\n",i+1);
			for (j=0;j<c2;j++) {
				printf("Element %d: ",j+1);
				scanf("%d",&e);
				m2[i][j]=e; } }
		printf("\nMatrix A\n\n");
		for (i=0;i<r1;i++) {
			for (j=0;j<c1;j++)
				printf("%d  ",m1[i][j]);
			printf("\n");
		}
		printf("\nMatrix B\n\n");
		for (i=0;i<r2;i++) {
			for (j=0;j<c2;j++)
				printf("%d  ",m2[i][j]);
			printf("\n");
		}
		int p[r1][c2];
```

```
            int temp_sum;
            for (i=0;i<r1;i++) {
                for (j=0;j<c2;j++) {
                    temp_sum = 0;
                    for (k=0;k<c1;k++)  // c1 or r2
                        temp_sum += m1[i][k]*m2[k][j];
                    p[i][j] = temp_sum;
                }
            }
            printf("\nAxB\n\n");
            for (i=0;i<r1;i++) {
                for (j=0;j<c2;j++) {
                    printf("%d ",p[i][j]);
                }
                printf("\n");
            }
        } else if (r1<=0 || c1<=0 || r2<=0 || c2<=0) {
            printf("\nRows and Columns should be greater than 0.\n");
        } else if (c1 != r2) {
            printf("\nMatrix Multiplication can't be performed for the given matrix
dimensions.\n");
        }
        return 0;
}
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/c_programs/exp_02$ cc matrix_multiplication.c
warspinix@Yuvaraj-Laptop:~/c_programs/exp_02$ ./a.out
Matrix A
No. of Rows: 2
No. of Columns: 3

Matrix B
No. of Rows: 3
No. of Columns: 2

Matrix A Elements

Row 1 Elements
Element 1: 1
Element 2: 2
Element 3: 3

Row 2 Elements
Element 1: 4
Element 2: 5
Element 3: 6

Matrix B Elements

Row 1 Elements
Element 1: 1
Element 2: 2

Row 2 Elements
Element 1: 3
Element 2: 7

Row 3 Elements
Element 1: 6
Element 2: 5
```

```
Matrix A

1  2  3
4  5  6

Matrix B

1  2
3  7
6  5

AxB

25 31
55 73
warspinix@Yuvaraj-Laptop:~/c_programs/exp_02$
```

TRANSPOSE OF A MATRIX

```
#include <stdio.h>
int main() {
	int rows, columns, i, j, e;
	printf("No. of Rows: ");
	scanf("%d",&rows);
	printf("No. of Columns: ");
	scanf("%d",&columns);
	int m[rows][columns];
	for (i=0;i<rows;i++) {
		printf("\nRow %d Elements\n",i+1);
		for (j=0;j<columns;j++) {
			printf("Element %d: ",j+1);
			scanf("%d",&e);
			m[i][j]=e;
		}
	}
	printf("\nMatrix\n");
	for (i=0;i<rows;i++) {
		for (j=0;j<columns;j++) {
			printf("%d ",m[i][j]);
		}
		printf("\n");
	}
	int t[columns][rows];
	for (i=0;i<columns;i++) {
		for (j=0;j<rows;j++) {
			t[i][j] = m[j][i];
		}
	}
	printf("\nTranspose\n");
	for (i=0;i<columns;i++) {
		for (j=0;j<rows;j++) {
			printf("%d ",t[i][j]);
		}
		printf("\n");
	}

	return 0;
}
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/c_programs/exp_02$ cc matrix_transpose.c
warspinix@Yuvaraj-Laptop:~/c_programs/exp_02$ ./a.out
No. of Rows: 6
No. of Columns: 7

Row 1 Elements
Element 1: 1
Element 2: 2
Element 3: 3
Element 4: 4
Element 5: 5
Element 6: 6
Element 7: 7

Row 2 Elements
Element 1: 8
Element 2: 9
Element 3: 8
Element 4: 7
Element 5: 6
Element 6: 5
Element 7: 4

Row 3 Elements
Element 1: 3
Element 2: 2
Element 3: 1
Element 4: 2
Element 5: 3
Element 6: 4
Element 7: 5

Row 4 Elements
Element 1: 6
Element 2: 7
Element 3: 8
Element 4: 9
Element 5: 8
Element 6: 7
```

```
Row 5 Elements
Element 1: 5
Element 2: 4
Element 3: 3
Element 4: 2
Element 5: 1
Element 6: 2
Element 7: 3

Row 6 Elements
Element 1: 4
Element 2: 5
Element 3: 6
Element 4: 7
Element 5: 8
Element 6: 9
Element 7: 8

Matrix
1 2 3 4 5 6 7
8 9 8 7 6 5 4
3 2 1 2 3 4 5
6 7 8 9 8 7 6
5 4 3 2 1 2 3
4 5 6 7 8 9 8

Transpose
1 8 3 6 5 4
2 9 2 7 4 5
3 8 1 8 3 6
4 7 2 9 2 7
5 6 3 8 1 8
6 5 4 7 2 9
7 4 5 6 3 8
warspinix@Yuvaraj-Laptop:~/c_programs/exp_02$
```

SYMMETRIC AND SKEW SYMMETRIC MATRICES

```
#include <stdio.h>
int main() {
	int n, i, j, e;
	printf("Dimension of Matrix: ");
	scanf("%d",&n);
	int m[n][n];
	for (i=0;i<n;i++) {
		printf("\nRow %d Elements\n",i+1);
		for (j=0;j<n;j++) {
			printf("Element %d: ",j+1);
			scanf("%d",&e);
			m[i][j]=e;
		}
	}
	printf("\nMatrix\n");
	for (i=0;i<n;i++) {
		for (j=0;j<n;j++)
			printf("%d ",m[i][j]);
		printf("\n");
	}
	int symmetric_check=1;
	for (i=1;i<n;i++) {
		for (j=0;j<i;j++) {
			if (i != j) {
				if (m[i][j] != m[j][i]) {
					symmetric_check = 0;
					break;
				}
			}
		}
		if (symmetric_check==0)
			break;
	}
	int skew_symmetric_check=1;
	for (i=1;i<n;i++) {
		for (j=0;j<=i;j++) {
			if (i==j) {
				if (m[i][j] != 0) {
					skew_symmetric_check=0;
					break;
				}
				else {
					if (m[i][j] != -m[j][i]) {
```

```
                                skew_symmetric_check=0;
                                break;
                            } } } }
            if (skew_symmetric_check==0)
                break;
        }
        if (symmetric_check==1)
            printf("\nThe matrix is symmetric.\n");
        else if (skew_symmetric_check==1)
            printf("\nThe matrix is skew symmetric.\n");
        else
            printf("\nThe matrix is neither symmetric nor skew skymmetric.\n");

        return 0;
}
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/c_programs/exp_02$ cc matrix_symmetry.c
warspinix@Yuvaraj-Laptop:~/c_programs/exp_02$ ./a.out
Dimension of Matrix: 3

Row 1 Elements
Element 1: 1
Element 2: 2
Element 3: 3

Row 2 Elements
Element 1: 2
Element 2: 2
Element 3: 4

Row 3 Elements
Element 1: 3
Element 2: 4
Element 3: 3

Matrix
1 2 3
2 2 4
3 4 3

The matrix is symmetric.
warspinix@Yuvaraj-Laptop:~/c_programs/exp_02$
```

```
warspinix@Yuvaraj-Laptop:~/c_programs/exp_02$ cc matrix_symmetry.c
warspinix@Yuvaraj-Laptop:~/c_programs/exp_02$ ./a.out
Dimension of Matrix: 4

Row 1 Elements
Element 1: 0
Element 2: 1
Element 3: 2
Element 4: 3

Row 2 Elements
Element 1: -1
Element 2: 0
Element 3: 4
Element 4: 5

Row 3 Elements
Element 1: -2
Element 2: -4
Element 3: 0
Element 4: 6

Row 4 Elements
Element 1: -3
Element 2: -5
Element 3: -6
Element 4: 0

Matrix
0 1 2 3
-1 0 4 5
-2 -4 0 6
-3 -5 -6 0

The matrix is skew symmetric.
warspinix@Yuvaraj-Laptop:~/c_programs/exp_02$
```

```
warspinix@Yuvaraj-Laptop:~/c_programs/exp_02$ cc matrix_symmetry.c
warspinix@Yuvaraj-Laptop:~/c_programs/exp_02$ ./a.out
Dimension of Matrix: 2

Row 1 Elements
Element 1: 1
Element 2: 2

Row 2 Elements
Element 1: 3
Element 2: 4

Matrix
1 2
3 4

The matrix is neither symmetric nor skew skymmetric.
warspinix@Yuvaraj-Laptop:~/c_programs/exp_02$
```

DETERMINANT OF A MATRIX

```
#include <stdio.h>

int minor(int matrix[2][2]) {
	return ((matrix[0][0]*matrix[1][1])-(matrix[0][1]*matrix[1][0]));
}

int determinant(int matrix[][3], int dimension) {
	if (dimension == 1)
		return matrix[0][0];
	else if (dimension == 2)
		return minor(matrix);

	int d = 0;
	int sign=1;
	int submatrix[2][2];

	int i, j, k;

	for (i=0;i<dimension;i++) {
		for (j=1;j<dimension;j++) {
			for (k=0;k<dimension;k++) {
				if (k<i)
					submatrix[j-1][k] = matrix[j][k];
				else if (k>i)
					submatrix[j-1][k-1] = matrix[j][k];
			}
		}
		sign = (i%2==0) ? 1 : -1;
		d += sign * matrix[0][i] * minor(submatrix);
	}
	return d;
}
int main() {
	int n, i, j, e;
	printf("Dimension of Matrix (1-3): ");
	scanf("%d",&n);

	if (n>0 && n<=3) {
		int m[n][n];

		for (i=0;i<n;i++) {
			printf("\nRow %d Elements\n",i+1);
			for (j=0;j<n;j++) {
```

```
                printf("Element %d: ",j+1);
                scanf("%d",&e);
                m[i][j]=e;
            }
        }
        printf("\nMatrix\n");

        for (i=0;i<n;i++) {
            for (j=0;j<n;j++) {
                printf("%d ",m[i][j]);
            }
            printf("\n");
        }

        int d=determinant(m,n);

        printf("\nThe determinant of the given matrix is %d.\n",d);
    } else {
        printf("\nPlease enter a dimension between 1 and 3 (inclusive of both).\n");
    }
    return 0;
}
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/c_programs/exp_02$ ./a.out
Dimension of Matrix (1-3): 1

Row 1 Elements
Element 1: 7

Matrix
7

The determinant of the given matrix is 7.
warspinix@Yuvaraj-Laptop:~/c_programs/exp_02$
```

```
warspinix@Yuvaraj-Laptop:~/c_programs/exp_02$ ./a.out
Dimension of Matrix (1-3): 0

Please enter a dimension between 1 and 3 (inclusive of both).
warspinix@Yuvaraj-Laptop:~/c_programs/exp_02$
```

```
warspinix@Yuvaraj-Laptop:~/c_programs/exp_02$ ./a.out
Dimension of Matrix (1-3): 2

Row 1 Elements
Element 1: 2
Element 2: 3

Row 2 Elements
Element 1: 1
Element 2: 2

Matrix
2 3
1 2

The determinant of the given matrix is 1.
warspinix@Yuvaraj-Laptop:~/c_programs/exp_02$
```

```
warspinix@Yuvaraj-Laptop:~/c_programs/exp_02$ ./a.out
Dimension of Matrix (1-3): 3

Row 1 Elements
Element 1: 4
Element 2: 3
Element 3: 2

Row 2 Elements
Element 1: 7
Element 2: 4
Element 3: 1

Row 3 Elements
Element 1: 3
Element 2: 3
Element 3: 5

Matrix
4 3 2
7 4 1
3 3 5

The determinant of the given matrix is -10.
warspinix@Yuvaraj-Laptop:~/c_programs/exp_02$
```

STRING OPERATIONS

```
#include <stdio.h>
#include <string.h>

int main() {
	char s[100];
	printf("Enter a String: ");
	scanf("%[^\n]99s",s);

	char copy[100];

	strcpy(copy,s);

	printf("Original String: %s\n",s);
	printf("Length = %zu\n",strlen(s)); // strlen returns value of type size t
	printf("Copied String: %s\n",copy);
	/* printf("Reversed: %s\n",strrev(s));
	printf("Lowercase: %s\n",strlwr(s));
	printf("Uppercase: %s\n",strupr(s)); */

	strcat(s,copy);
	printf("Original String after Concatenation: %s\n",s);

	if (strcmp(s,copy) == 0)
		printf("The two strings are the same.\n");
	else
		printf("The two strings are different.\n");

	return 0;
}
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/c_programs/exp_02$ cc string_operations.c
warspinix@Yuvaraj-Laptop:~/c_programs/exp_02$ ./a.out
Enter a String: Yuvaraj
Original String: Yuvaraj
Length = 7
Copied String: Yuvaraj
Original String after Concatenation: YuvarajYuvaraj
The two strings are different.
warspinix@Yuvaraj-Laptop:~/c_programs/exp_02$
```

STRING CONCATENATION

```
#include <stdio.h>
int main() {

	char s1[100], s2[100];
	printf("Enter String 1: ");
	scanf("%99s",s1);
	printf("Enter String 2: ");
	scanf("%99s",s2);

	char *p_s1 = s1;
	char *p_s2 = s2;

	while (*p_s1 != '\0')
		p_s1++;

	while (*p_s2 != '\0') {
		*p_s1 = *p_s2;
		p_s1++;
		p_s2++;
	}

	*p_s1 = '\0';

	printf("Concatenated String: %s\n", s1);

	return 0;
}
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/c_programs/exp_02$ cc string_concatenate.c
warspinix@Yuvaraj-Laptop:~/c_programs/exp_02$ ./a.out
Enter String 1: Yuva
Enter String 2: raj
Concatenated String: Yuvaraj
warspinix@Yuvaraj-Laptop:~/c_programs/exp_02$
```

RECURSIVE AND ITERATIVE FUNCTIONS

(i) FACTORIAL

```
#include <stdio.h>
int factorial_iterative(int n) {
	if (n<0) {
		return -1;
	} else if (n==0) {
		return 1;
	} else {
		int i, f=1;
		for (i=1;i<=n;i++)
			f = f*i;
		return f;
	}
}
int factorial_recursive(int n) {
	if (n<0)
		return -1;
	else if (n==0)
		return 1;
	else
		return n*factorial_recursive(n-1);
}
int main() {
	int n, fi, fr;
	printf("Enter a Number: ");
	scanf("%d",&n);
	fi = factorial_iterative(n);
	fr = factorial_recursive(n);
	printf("\nFactorial of %d (iterative) = %d\n",n,fi);
	printf("Factorial of %d (recursive) = %d\n",n,fr);
	return 0;
}
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/c_programs/exp_02$ cc factorial_ri.c
warspinix@Yuvaraj-Laptop:~/c_programs/exp_02$ ./a.out
Enter a Number: 7

Factorial of 7 (iterative) = 5040
Factorial of 7 (recursive) = 5040
warspinix@Yuvaraj-Laptop:~/c_programs/exp_02$
```

(ii) SUM OF FIRST *n* NATURAL NUMBERS

```
#include <stdio.h>
int sum_iterative(int);
int sum_recursive(int);
int main() {
	int n, si, sr;
	printf("Enter a Number: ");
	scanf("%d",&n);
	si = sum_iterative(n);
	sr = sum_recursive(n);
	printf("\nSum of first %d natural numbers (iterative) = %d\n",n,si);
	printf("Sum of first %d natural numbers (recursive) = %d\n",n,sr);
	return 0;
}
int sum_iterative(int n) {
	if (n<0) {
		return -1;
	} else {
		int i, s=0;
		for (i=1;i<=n;i++)
			s = s+i;
		return s;
	}
}
int sum_recursive(int n) {
	if (n<0)
		return -1;
	else if (n==0)
		return 0;
	else
		return n+sum_recursive(n-1);
}
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/c_programs/exp_02$ cc sum_of_first_n_ri.c
warspinix@Yuvaraj-Laptop:~/c_programs/exp_02$ ./a.out
Enter a Number: 10

Sum of first 10 natural numbers (iterative) = 55
Sum of first 10 natural numbers (recursive) = 55
warspinix@Yuvaraj-Laptop:~/c_programs/exp_02$
```

(iii) FIBONACCI SERIES

```
#include <stdio.h>

void fibonacci_iterative(int);
int fibonacci(int);
void fibonacci_recursive(int);

int main() {
	int n;
	printf("Enter the Number of Terms: ");
	scanf("%d",&n);
	printf("\nIterative: ");
	fibonacci_iterative(n);
	printf("\nRecursive: ");
	fibonacci_recursive(n);
	return 0;
}
void fibonacci_iterative(int n) {
	int a=0, b=1;
	if (n>0) {
		if (n==1) {
			printf("Fibonacci Series upto 1 Term\n\n");
			printf("0");
		} else if (n==2) {
			printf("Fibonacci Series upto 2 Terms\n\n");
			printf("0 1");
		} else {
			printf("Fibonacci Series upto %d Terms\n\n",n);
			printf("%d %d ",a,b);
			int i, c;
			for (i=0;i<n-2;i++) {
				c=a+b;
				printf("%d ",c);
				a=b;
				b=c;
			}
		}
	} else {
		printf("The number should be above 0.");
	}
	printf("\n");
}
int fibonacci(int n) {
	if (n==1)
```

```
        return 0;
    else if (n==2)
        return 1;
    else
        return fibonacci(n-1)+fibonacci(n-2);
}
void fibonacci_recursive(int n) {
    if (n>0) {
        if (n==1) {
            printf("Fibonacci Series upto 1 Term\n\n");
            printf("%d",fibonacci(n));
        } else {
            printf("Fibonacci Series upto %d Terms\n\n",n);
            int i;
            for (i=1;i<=n;i++) {
                printf("%d ",fibonacci(i));
            }
        }
    } else {
        printf("The number should be above 0.");
    }
    printf("\n");
}
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/c_programs/exp_02$ cc fibonacci_ri.c
warspinix@Yuvaraj-Laptop:~/c_programs/exp_02$ ./a.out
Enter the Number of Terms: 7

Iterative: Fibonacci Series upto 7 Terms

0 1 1 2 3 5 8

Recursive: Fibonacci Series upto 7 Terms

0 1 1 2 3 5 8
warspinix@Yuvaraj-Laptop:~/c_programs/exp_02$ ./a.out
Enter the Number of Terms: 0

Iterative: The number should be above 0.

Recursive: The number should be above 0.
warspinix@Yuvaraj-Laptop:~/c_programs/exp_02$
```

LINEAR SEARCH

```
#include <stdio.h>
int main() {
	int n, i, e;
	printf("Enter the Number of Elements: ");
	scanf("%d",&n);
	if (n>0) {
		int arr[n];
		printf("\n");
		for (i=0;i<n;i++) {
			printf("Element %d: ",i+1);
			scanf("%d",&e);
			arr[i]=e; }
		printf("\nEnter the Element to be Searched for: ");
		scanf("%d",&e);
		int found=0, pos;
		for (pos=0;pos<n;pos++) {
			if (arr[pos]==e) {
				found=1;
				break; } }
		if (found==1)
			printf("\n%d is found at position %d of the array.\n",e,pos);
		else
			printf("\n%d is NOT present in the array.\n",e);
	} else
		printf("\nThe number of elements should be greater than 0.\n");
	return 0;
}
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/c_programs/exp_02$ cc linear_search.c
warspinix@Yuvaraj-Laptop:~/c_programs/exp_02$ ./a.out
Enter the Number of Elements: 5

Element 1: 1
Element 2: 3
Element 3: 5
Element 4: 7
Element 5: 9

Enter the Element to be Searched for: 7

7 is found at position 3 of the array.
warspinix@Yuvaraj-Laptop:~/c_programs/exp_02$ ./a.out
Enter the Number of Elements: 5

Element 1: 1
Element 2: 3
Element 3: 5
Element 4: 7
Element 5: 9

Enter the Element to be Searched for: 2

2 is NOT present in the array.
warspinix@Yuvaraj-Laptop:~/c_programs/exp_02$
```

BUBBLE SORT

```
#include <stdio.h>
int main() {
	int n, i, e;
	printf("Number of Elements: ");
	scanf("%d",&n);
	if (n>0) {
		int arr[n];
		printf("\n");
		for (i=0;i<n;i++) {
			printf("Element %d: ",i+1);
			scanf("%d",&e);
			arr[i]=e; }
		printf("\nMy Array\n");
		for (i=0;i<n;i++)
			printf("%d ",arr[i]);
		int s, temp;
		for (s=0;s<n;s++) {
			for (i=0;i<n-1;i++) {
				if (arr[i]>arr[i+1]) {
					temp=arr[i];
					arr[i]=arr[i+1];
					arr[i+1]=temp; } } }
		printf("\n\nSorted Array\n");
		for (i=0;i<n;i++)
			printf("%d ",arr[i]);
		printf("\n");
	} else {
		printf("The number of elements should be greater than 0.");
	}
	return 0;
}
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/c_programs/exp_02$ cc bubble_sort.c
warspinix@Yuvaraj-Laptop:~/c_programs/exp_02$ ./a.out
Number of Elements: 7

Element 1: 9
Element 2: 7
Element 3: 5
Element 4: 3
Element 5: 1
Element 6: 2
Element 7: 4

My Array
9 7 5 3 1 2 4

Sorted Array
1 2 3 4 5 7 9
warspinix@Yuvaraj-Laptop:~/c_programs/exp_02$
```

COMMAND LINE ARGUMENTS

```
#include <stdio.h>
int main (int argc, char *argv[]) {
	printf("\n %d, %s %s \n\n",argc, argv[0], argv[1]);
	return 0;
}
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/c_programs/exp_02$ cc command_line_arguments.c
warspinix@Yuvaraj-Laptop:~/c_programs/exp_02$ ./a.out

 1, ./a.out (null)

warspinix@Yuvaraj-Laptop:~/c_programs/exp_02$ ./a.out First_Argument

 2, ./a.out First_Argument

warspinix@Yuvaraj-Laptop:~/c_programs/exp_02$
```

Here, argv[0] holds the name of the program, and argv[1] is a pointer to the first command line argument supplied. Naturally, *argv[n] is the last argument. If no arguments are supplied, argc will be 1, and if you pass one argument, argc is set at 2.

Evaluation Criteria	**Observation**	**Record**
Ability for problem definition and realisation	…../**10**	…../**10**
Ability to design and analyse	…../**10**	…../**10**
Ability to implement and validate	…../**10**	…../**10**

RESULT

Various problems and applications pertaining to single dimensional and multi-dimensional arrays, and also functions were successfully explored and worked out.

Exp. No: 4
Date: 05/10/23

SEARCHING AND SORTING TECHNIQUES

AIM

To implement linear search, binary search, insertion sort, merge sort, and quick sort.

HEADER FILE - *arrio.h*

```
void inputArray(int arr[], int length) {
	int i;
	printf("\nEnter Array Elements\n");
	for (i=0; i<length; i++) {
		printf("Element %d: ",i+1);
		scanf("%d",&arr[i]);
	}
}
void printArray(int arr[], int length) {
	int i;
	printf("\nYour Array\n");
	for (i=0; i<length; i++)
		printf("%d ",arr[i]);
	printf("\n");
}
void printSortedArray(int arr[], int length) {
	int i;
	printf("\nYour Sorted Array\n");
	for (i=0; i<length; i++)
		printf("%d ",arr[i]);
	printf("\n");
}
```

HEADER FILE - *search.h*

```
int linearSearch(int arr[], int length, int element) {
	int pos;
	for (pos=0; pos<length; pos++) {
		if (arr[pos]==element)
			return pos;
	}
	return -1;
}
int binarySearch(int arr[], int element, int low, int high) {
	if (low>high)
		return -1;
	int mid=(low+high)/2;
	if (arr[mid]<element)
		return binarySearch(arr, element, mid+1, high);
	else if (arr[mid]>element)
		return binarySearch(arr, element, low, mid-1);
	int i;
	for (i=mid-1;i>=0;i--) {
		if (arr[i]==arr[mid])
			mid--;
		else
			break;
	}
	return mid;
}
int getElement() {
	int element;
	printf("\nEnter the Search Element: ");
	scanf("%d",&element);
	return element;
}
void search(int arr[], int length, int element, int c)  {
	int p;
	if (c==1)
		p=linearSearch(arr, length, element);
	else if (c==2)
		p=binarySearch(arr, element, 0, length-1);
	if (p>=0)
		printf("\n%d is found at position %d.\n",element,p+1);
	else
		printf("\n%d is NOT present in the array.\n",element);
}
```

HEADER FILE - *insertion_sort.h*

```
void insertionSort(int arr[], int length) {
	int i, j, key;
	for (i=1;i<length;i++) {
		key=arr[i];
		j=i-1;
		while (j>=0 && arr[j]>key) {
			arr[j+1]=arr[j];
			j--;
		}
		arr[j+1]=key;
	}
}
```

HEADER FILE - *quick_sort.h*

```
void swap(int *x, int *y) {
	int temp;
	temp=*x;
	*x=*y;
	*y=temp;
}
int partition (int arr[], int low, int high) {
	int pivot=arr[high]; // Pivot is last element
	int i=low-1,j;
	for (j=low; j<=high-1;j++) {
		if (arr[j]<pivot) {
			i++;
			swap(&arr[i], &arr[j]);
		}
	}
	swap(&arr[i+1], &arr[high]);
	return i+1;
}
void quickSort(int arr[], int low, int high) {
	if (low<high) {
		int pi=partition(arr, low, high);
		quickSort(arr, low, pi-1);
		quickSort(arr, pi+1, high);
	}
}
```

HEADER FILE - *merge_sort.h*

```
void merge(int arr[], int lb, int mid, int ub) {
	int temp[ub-lb+1];
	int i=lb, j=mid+1, k=0;
	while (i<=mid && j<=ub) {
		if (arr[i]<=arr[j]) {
			temp[k]=arr[i];
			i++;
		} else {
			temp[k]=arr[j];
			j++;
		}
		k++;
	}
	if (i>mid) {
		while (j<=ub) {
			temp[k]=arr[j];
			j++;
			k++;
		}
	} else {
		while (i<=mid) {
			temp[k]=arr[i];
			i++;
			k++;
		}
	}
	int e;
	for (e=0; e<k; e++)
		arr[lb+e]=temp[e];
}
void mergeSort(int arr[], int lb, int ub) {
	if (lb<ub) {
		int mid=lb+(ub-lb)/2;
		mergeSort(arr, lb, mid);
		mergeSort(arr, mid+1, ub);
		merge(arr, lb, mid, ub);
	}
}
```

PROGRAM FILE - *menu_driven.c*

```
#include <stdio.h>
#include "arrio.h"
#include "search.h"
#include "insertion_sort.h"
#include "merge_sort.h"
#include "quick_sort.h"

int main() {

	int choice, length, element;
	do {
		printf("\nEnter 0 to Exit\n");
		printf("Enter 1 for Linear Search\n");
		printf("Enter 2 for Binary Search\n");
		printf("Enter 3 for Insertion Sort\n");
		printf("Enter 4 for Merge Sort\n");
		printf("Enter 5 for Quick Sort\n");

		printf("\nEnter your Choice: ");
		scanf("%d",&choice);

		if (choice==0)
			break;
		if (choice<0 || choice>5) {
			printf("\nInvalid Input.\n");
			continue;
		}

		printf("\nLength of the Array: ");
		scanf("%d",&length);
		int arr[length];

		inputArray(arr, length);
		printArray(arr, length);

		switch (choice) {
			case 1:
				element = getElement();
				search(arr, length, element, 1);
				break;
			case 2:
				quickSort(arr, 0, length-1);
				printSortedArray(arr, length);
```

```
                element = getElement();
                search(arr, length, element, 2);
                break;
            case 3:
                insertionSort(arr, length);
                printSortedArray(arr, length);
                break;
            case 4:
                mergeSort(arr, 0, length-1);
                printSortedArray(arr, length);
                break;
            case 5:
                quickSort(arr, 0, length-1);
                printSortedArray(arr, length);
                break;
        }
    } while (1);
    printf("\nProcess Exited.\n");
    return 0;
}
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/c_programs/exp_04$ cc menu_driven.c
warspinix@Yuvaraj-Laptop:~/c_programs/exp_04$ ./a.out

Enter 0 to Exit
Enter 1 for Linear Search
Enter 2 for Binary Search
Enter 3 for Insertion Sort
Enter 4 for Merge Sort
Enter 5 for Quick Sort

Enter your Choice: 1

Length of the Array: 4

Enter Array Elements
Element 1: 73
Element 2: 37
Element 3: 33
Element 4: 21

Your Array
73 37 33 21

Enter the Search Element: 33

33 is found at position 3.
```

```
Enter 0 to Exit
Enter 1 for Linear Search
Enter 2 for Binary Search
Enter 3 for Insertion Sort
Enter 4 for Merge Sort
Enter 5 for Quick Sort

Enter your Choice: 2

Length of the Array: 4

Enter Array Elements
Element 1: 73
Element 2: 37
Element 3: 33
Element 4: 21

Your Array
73 37 33 21

Your Sorted Array
21 33 37 73

Enter the Search Element: 37

37 is found at position 3.
```

```
Enter 0 to Exit
Enter 1 for Linear Search
Enter 2 for Binary Search
Enter 3 for Insertion Sort
Enter 4 for Merge Sort
Enter 5 for Quick Sort

Enter your Choice: 3

Length of the Array: 5

Enter Array Elements
Element 1: 30
Element 2: 21
Element 3: 127
Element 4: 103
Element 5: 243

Your Array
30 21 127 103 243

Your Sorted Array
21 30 103 127 243
```

```
Enter 0 to Exit
Enter 1 for Linear Search
Enter 2 for Binary Search
Enter 3 for Insertion Sort
Enter 4 for Merge Sort
Enter 5 for Quick Sort

Enter your Choice: 4

Length of the Array: 4

Enter Array Elements
Element 1: 1043
Element 2: 845
Element 3: 1270
Element 4: 345

Your Array
1043 845 1270 345

Your Sorted Array
345 845 1043 1270
```

```
Enter 0 to Exit
Enter 1 for Linear Search
Enter 2 for Binary Search
Enter 3 for Insertion Sort
Enter 4 for Merge Sort
Enter 5 for Quick Sort

Enter your Choice: 5

Length of the Array: 5

Enter Array Elements
Element 1: 204
Element 2: 127
Element 3: 1120
Element 4: 3
Element 5: 7

Your Array
204 127 1120 3 7

Your Sorted Array
3 7 127 204 1120
```

```
Enter 0 to Exit
Enter 1 for Linear Search
Enter 2 for Binary Search
Enter 3 for Insertion Sort
Enter 4 for Merge Sort
Enter 5 for Quick Sort

Enter your Choice: 7

Invalid Input.

Enter 0 to Exit
Enter 1 for Linear Search
Enter 2 for Binary Search
Enter 3 for Insertion Sort
Enter 4 for Merge Sort
Enter 5 for Quick Sort

Enter your Choice: 0

Process Exited.
warspinix@Yuvaraj-Laptop:~/c_programs/exp_04$
```

Evaluation Criteria	Observation	Record
Ability for problem definition and realisation	…../**10**	…../**10**
Ability to design and analyse	…../**10**	…../**10**
Ability to implement and validate	…../**10**	…../**10**

RESULT

The various searching and sorting functions were defined in header files and implemented successfully as a menu driven program in C.

Exp. No: 5
Date: 12/10/23

IMPLEMENTATION OF STRUCTURE, UNION, AND ENUMERATION DATA TYPES

AIM

To understand the concepts of structure, union, and enumeration data types.

STRUCTURES

```
struct date {
	int day;
	int month;
	int year;
}d;

struct Student {
	int reg_no;
	char name[20];
	char book[30];
	struct date borrow_date;
	struct date return_date;
	struct date actual_return_date;
	int fine;
}s;

void getBorrowDate() {
	printf("Date: ");
	scanf("%d",&s.borrow_date.day);
	printf("Month: ");
	scanf("%d",&s.borrow_date.month);
	printf("Year: ");
	scanf("%d",&s.borrow_date.year);
}

void displayBorrowDate() {
	printf("%d-%d-%d\n",s.borrow_date.day,s.borrow_date.month,s.borrow_date.year);
}

void returnDate() {
	int bd=s.borrow_date.day, bm=s.borrow_date.month, by=s.borrow_date.year;
```

```
        s.return_date.day=bd+7;
        s.return_date.month=bm;
        s.return_date.year=by;


        if (s.return_date.day>31) {
                s.return_date.day%=31;
                s.return_date.month++;
                if (s.return_date.month>12) {
                        s.return_date.month%=31;
                        s.return_date.year++;
                }
        }
}

void displayReturnDate() {
        printf("%d-%d-%d\n",s.return_date.day,s.return_date.month,s.return_date.year);
}

void getActualReturnDate() {
        printf("Date: ");
        scanf("%d",&s.actual_return_date.day);
        printf("Month: ");
        scanf("%d",&s.actual_return_date.month);
        printf("Year: ");
        scanf("%d",&s.actual_return_date.year);
}

void displayActualReturnDate() {

printf("%d-%d-%d\n",s.actual_return_date.day,s.actual_return_date.month,s.actual_return_da
te.year);
}

void sRegister() {
        printf("\nRegister Number: ");
        scanf("%d",&s.reg_no);
        printf("Name: ");
        scanf("%s",s.name);
        s.fine=0;
        printf("\nRegistration Complete.\n\n");
}

void borrow() {
        if (strlen(s.name) != 0) {
                if (strlen(s.book) == 0) {
```

```
                printf("\nName of the Book: ");
                scanf("%s",s.book);
                printf("Borrow Date\n");
                getBorrowDate();
                returnDate();
                printf("\n");
            } else
                printf("\nYou have to return your current book to borrow another.\n\n");

        } else
            printf("\nYou have to register in order to borrow a book.\n\n");
}

void bookReturn() {
        if (strlen(s.name) != 0) {
            if (strlen(s.book) != 0) {
                printf("\nName: %s\n",s.name);
                printf("Register Number: %d\n",s.reg_no);
                printf("Book to be Returned: %s\n",s.book);
                printf("Return Date\n");
                getActualReturnDate();
                printf("\n");
                *s.book='\0';
            } else
                printf("\nYou have to borrow a book first.\n\n");
        } else
            printf("\nYou have to register.\n\n");
}

void calculateFine() {
        if (strlen(s.name) != 0) {
            int rd=s.return_date.day, rm=s.return_date.month, ry=s.return_date.year;
            int ad=s.actual_return_date.day, am=s.actual_return_date.month,
ay=s.actual_return_date.year;
            if (ay<ry || (ay==ry && (am<rm || (am==rm && ad<=rd)))) {
                s.fine+=0;
            } else {
                int fine=0;
                if (ay==ry && am==rm)
                    fine=ad-rd;
                else if (ay==ry && am>rm)
                    fine = (am - rm) * 30 + (ad - rd);
                else if (ay > ry)
                    fine = (ay - ry) * 365 + (am - rm) * 30 + (ad - rd);
                s.fine += fine;
            }
```

```
		printf("\nName: %s\n",s.name);
		printf("Fine = %d\n\n",s.fine);
	} else
		printf("\nYou have to register.\n\n");
}

void display() {
	if (strlen(s.name) != 0) {
		printf("\nMember Details\n\n");
		printf("Register Number: %d\n",s.reg_no);
		printf("Name: %s\n\n",s.name);
		if (strlen(s.book) != 0) {
			printf("Book Borrowed: %s\n",s.book);
			printf("Date Borrowed: ");
			displayBorrowDate();
			printf("Expected Return Date: ");
			displayReturnDate();
			printf("\n");
		}
	} else
		printf("\nYou have to register.\n\n"); }

int main() {
	int choice;
	do {
		printf("What do you want to do?\n\n");
		printf("0-Exit\n1-Register\n2-Borrow\n3-Return\n4-Calculate Fine\n5-Display
Details\n\n");
		printf("Enter Your Choice: ");
		scanf("%d",&choice);

		if (choice==0) {
			printf("\nProcess Exited.\n");
			break;
		}
		if (choice<0 || choice>5) {
			printf("\nInvalid.\n\n");
			continue;
		}

		switch (choice) {

			case 1:
				sRegister();
				break;
			case 2:
```

```
                borrow();
                break;
            case 3:
                bookReturn();
                break;
            case 4:
                calculateFine();
                break;
            case 5:
                display();
                break;

        }
    } while (1);
    return 0;
}
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/c_programs/exp_05$ cc struct.c
warspinix@Yuvaraj-Laptop:~/c_programs/exp_05$ ./a.out
What do you want to do?

0-Exit
1-Register
2-Borrow
3-Return
4-Calculate Fine
5-Display Details

Enter Your Choice: 5

You have to register.

What do you want to do?

0-Exit
1-Register
2-Borrow
3-Return
4-Calculate Fine
5-Display Details

Enter Your Choice: 1

Register Number: 2022506079
Name: Yuvaraj

Registration Complete.
```

```
What do you want to do?

0-Exit
1-Register
2-Borrow
3-Return
4-Calculate Fine
5-Display Details

Enter Your Choice: 2

Name of the Book: Programming
Borrow Date
Date: 13
Month: 10
Year: 2023

What do you want to do?

0-Exit
1-Register
2-Borrow
3-Return
4-Calculate Fine
5-Display Details

Enter Your Choice: 5

Member Details

Register Number: 2022506079
Name: Yuvaraj

Book Borrowed: Programming
Date Borrowed: 13-10-2023
Expected Return Date: 20-10-2023
```

```
What do you want to do?

0-Exit
1-Register
2-Borrow
3-Return
4-Calculate Fine
5-Display Details

Enter Your Choice: 3

Name: Yuvaraj
Register Number: 2022506079
Book to be Returned: Programming
Return Date
Date: 27
Month: 10
Year: 2023

What do you want to do?

0-Exit
1-Register
2-Borrow
3-Return
4-Calculate Fine
5-Display Details

Enter Your Choice: 4

Name: Yuvaraj
Fine = 7
```

```
What do you want to do?

0-Exit
1-Register
2-Borrow
3-Return
4-Calculate Fine
5-Display Details

Enter Your Choice: 5

Member Details

Register Number: 2022506079
Name: Yuvaraj

What do you want to do?

0-Exit
1-Register
2-Borrow
3-Return
4-Calculate Fine
5-Display Details

Enter Your Choice: 0

Process Exited.
warspinix@Yuvaraj-Laptop:~/c_programs/exp_05$
```

ENUMERATIONS

```
#include <stdio.h>
enum month {January, February, March, April, May, June, July, August, September, October, November, December};
int main() {

printf("January: %d\nFebruary: %d\nMarch: %d\nApril: %d\nMay: %d\nJune: %d\nJuly: %d\nAugust: %d\nSeptember: %d\nOctober: %d\nNovember: %d\nDecember: %d\n",January, February, March, April, May, June, July, August, September, October, November, December);
	return 0;
}
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/c_programs/exp_05$ cc enum.c
warspinix@Yuvaraj-Laptop:~/c_programs/exp_05$ ./a.out
January: 0
February: 1
March: 2
April: 3
May: 4
June: 5
July: 6
August: 7
September: 8
October: 9
November: 10
December: 11
warspinix@Yuvaraj-Laptop:~/c_programs/exp_05$
```

UNIONS

```
#include <stdio.h>
union Student {
	int reg_no;
	char name[30];
	char dept[3];
}s;
int main() {
	printf("Register No: ");
	scanf("%d",&s.reg_no);
	printf("Register No: %d\nName: %d\nDept: %d\n",s.reg_no,s.name,s.dept);
	printf("\nName: ");
	scanf("%s",s.name);
	printf("Register No: %d\nName: %s\nDept: %s\n",s.reg_no,s.name,s.dept);
	printf("\nDepartment: ");
	scanf("%s",s.dept);
	printf("Register No: %d\nName: %s\nDept: %s\n",s.reg_no,s.name,s.dept);
	return 0;
}
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/c_programs/exp_05$ ./a.out
Register No: 2022506079
Register No: 2022506079
Name: -1968750528
Dept: -1968750528

Name: Yuvaraj
Register No: 1635153241
Name: Yuvaraj
Dept: Yuvaraj

Department: IT
Register No: 1627411529
Name: IT
Dept: IT
warspinix@Yuvaraj-Laptop:~/c_programs/exp_05$
```

Evaluation Criteria	Observation	Record
Ability for problem definition and realisation	…../**10**	…../**10**
Ability to design and analyse	…../**10**	…../**10**
Ability to implement and validate	…../**10**	…../**10**

RESULT

A simple library management system was successfully implemented using structures, and other user-defined data types such as enumerations and unions were also briefly experimented with.

Exp. No: 6
Date: 2/11/23

IMPLEMENTATION OF DYNAMIC MEMORY ALLOCATION AND FILE HANDLING

AIM

To implement dynamic memory allocation in structures and the various file handling operations.

DYNAMIC MEMORY ALLOCATION AND FILE WRITING

```
#include <stdio.h>
#include <stdlib.h>
struct electricity {
	int units;
	float cost;
	char name[20];
};
float calculate(int units) {
	float cost;
	if (units>0 && units<=500) {
		if (units<100)
			cost=0;
		else if (units<200)
			cost=(units-100)*2.25;
		else if (units<400)
			cost=225+(units-200)*4.5;
		else if (units<=500)
			cost=1125+(units-400)*6;
		return cost;
	} else
		return -1;
}
int main() {
	FILE * fp;
	fp=fopen("EB.txt","w");
	struct electricity *p;
	int n,i,temp;
	printf("No. of Users: ");
	scanf("%d",&n);
```

```
    p=(struct electricity *)malloc(n*sizeof(struct electricity));
    for (i=0;i<n;i++) {
        printf("\nUser %d\n",i+1);
        printf("Name: ");
        scanf("%s",(p+i)->name);
        printf("No. of Units: ");
        scanf("%d",&(p+i)->units);
    }
    printf("\n");
    for (i=0;i<n;i++) {
        temp=(p+i)->units;
        (p+i)->cost=calculate(temp);
        printf("Name: %s\n",(p+i)->name);
        if ((p+i)->cost>=0)
            printf("Bill = Rs. %.2f\n",(p+i)->cost);
        else
            printf("Entered Value is Invalid.\n");
        printf("-----------------------------------\n");
        fprintf(fp,"%s\t\t%d\t\t%.2f\n",(p+i)->name,(p+i)->units,(p+i)->cost);
    }
    fclose(fp);
    free(p);
}
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/c_programs/exp_06$ cc dynamic.c
warspinix@Yuvaraj-Laptop:~/c_programs/exp_06$ ./a.out
No. of Users: 3

User 1
Name: BK
No. of Units: 222

User 2
Name: Babu
No. of Units: 444

User 3
Name: Yuvaraj
No. of Units: 333

Name: BK
Bill = Rs. 324.00
-----------------------------------
Name: Babu
Bill = Rs. 1389.00
-----------------------------------
Name: Yuvaraj
Bill = Rs. 823.50
-----------------------------------
warspinix@Yuvaraj-Laptop:~/c_programs/exp_06$
```

FILE READING

```
#include <stdio.h>
#include <stdlib.h>

int main() {
	FILE * fp;
	fp=fopen("EB.txt","r");
	char name[20];
	int units;
	float cost;
	printf("NAME\t\tUNITS\t\tTOTAL COST\n");
	while(!(feof(fp))) {
		fscanf(fp,"%s\t\t%d\t\t%f\n",name, &units, &cost);
		printf("%s\t\t%d\t\t%.2f\n",name, units, cost);
	}
}
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/c_programs/exp_06$ cc readfile.c
warspinix@Yuvaraj-Laptop:~/c_programs/exp_06$ ./a.out
NAME            UNITS           TOTAL COST
BK              222             324.00
Babu            444             1389.00
Yuvaraj         333             823.50
warspinix@Yuvaraj-Laptop:~/c_programs/exp_06$
```

Evaluation Criteria	Observation	Record
Ability for problem definition and realisation	…../**10**	…../**10**
Ability to design and analyse	…../**10**	…../**10**
Ability to implement and validate	…../**10**	…../**10**

RESULT

A simple application was implemented using dynamic memory allocation of structures, and the struct data was successfully written into and read from a file.

Exp. No: 7
Date: 16/11/23

IMPLEMENTATION OF LINKED LIST

AIM

To implement linked list operations.

INCLUDING REQUIRED HEADER FILES

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int len=0;
```

CREATING HEAD NODE

```
struct node * head;
struct node **headRef=&head;
struct node * last;

void createHeadNode() {
	head=(struct node *)malloc(sizeof(struct node));
}

void addData(struct node *new) {
	printf("Name: ");
	scanf("%s",new->name);
	printf("Register Number: ");
	scanf("%d",&new->rno);
	printf("Age: ");
	scanf("%d",&new->age);
	len++;
}
```

DISPLAYING LINKED LIST

```
void display() {
	int count=0;
	last=*headRef;
	printf("\nNode\t|Student Name\t|No\t|Age\t|Address\t|Next Address\n");
	while (last!=NULL) {
		if (count==0)
			printf("Head\t");
		else
			printf("Node %d\t",count);
		if (strlen(last->name)<=6)
			printf("|%s\t\t",last->name);
		else
			printf("|%s\t",last->name);
		printf("|%d\t|%d\t|%p\t",last->rno,last->age,last);
		if (last->next != NULL)
			printf("|%p\n",last->next);
		else
			printf("|NULL\n");
		last=last->next;
		count++;
	}
	printf("\nTotal Number of Nodes = %d\n",len);
}
```

APPENDING NEW NODES

```
void append() {
	struct node *new=(struct node *)malloc(sizeof(struct node));
	addData(new);
	last=*headRef;
	while (last->next!=NULL)
		last=last->next;
	last->next=new;
}
```

PUSHING NEW NODES

```
void push() {
	struct node *new=(struct node *)malloc(sizeof(struct node));
	addData(new);
	new->next=*headRef;
	*headRef=new;
}
```

INSERTING NEW NODES

```
void insert(int pos) {
	if (pos==0) {
		push();
		return;
	}
	else if (pos==len) {
		append();
		return;
	}
	struct node *new=(struct node *)malloc(sizeof(struct node));
	int count=0;
	last=*headRef;
	while (count<=len-2) {
		if (pos-1==count) {
			addData(new);
			new->next=last->next;
			last->next=new;
			return;
		}
		last=last->next;
		count++;
	}
}
```

UPDATING NODE DATA

```
 void updateName(int pos) {
	int count=0;
	last=*headRef;
	while (last!=NULL) {
		if (pos==count) {
			printf("Old Name: %s\n",last->name);
			printf("Updated Name: ");
			scanf("%s",last->name);
		}
		last=last->next;
		count++;
	}
}
```

```
void updateNo(int pos) {
	int count=0;
	last=*headRef;
	while (last!=NULL) {
		if (pos==count) {
			printf("Old Register Number: %d\n",last->rno);
			printf("Updated Register Number: ");
			scanf("%d",&last->rno);
		}
		last=last->next;
		count++;
	}
}

void updateAge(int pos) {
	int count=0;
	last=*headRef;
	while (last!=NULL) {
		if (pos==count) {
			printf("Old Age: %d\n",last->age);
			printf("Updated Age: ");
			scanf("%d",&last->age);
		}
		last=last->next;
		count++;
	}
}
```

DELETING NODE BY POSITION

```
void deleteByPosition(int pos) {
	struct node *node_to_delete;
	last=*headRef;
	if (pos==0) {
		node_to_delete=last;
		*headRef=last->next;
	} else {
		int count=0;
		while (count<=len-2) {
			if (pos-1==count) {
				node_to_delete=last->next;
				last->next=node_to_delete->next;
				break;
			}
```

```
                last=last->next;
                count++;
            }
        }
        free(node_to_delete);
        len--;
}
```

DELETING NODE BY REGISTER NUMBER

```
void displayNo() {
        printf("\nAll Available Students\n");
        last=*headRef;
        int i=1;
        while (last!=NULL) {
            printf("%d) %d\n",i,last->rno);
            i++;
            last=last->next;
        }
}
int deleteByNo(int rno) {
        struct node *node_to_delete;
        last=*headRef;
        int flag=0;
        if (rno==last->rno) {
            node_to_delete=last;
            *headRef=last->next;
            flag=1;
        } else {
            struct node *next_node;
            while (last->next!=NULL) {
                next_node=last->next;
                if (rno==next_node->rno) {
                    node_to_delete=next_node;
                    next_node=next_node->next;
                    flag=1;
                }
                last=last->next;
            }
        }
        if (flag) {
            free(node_to_delete);
            len--;
        }
        return flag;
}
```

MENU DRIVEN PROGRAM

```
int main() {

	createHeadNode();
	printf("Head Node Details\n");
	addData(head);

	int choice, pos;

	while (1) {

		printf("\n0. Exit\n1. Display\n2. Append\n3. Push\n4. Insert\n5. Update Name\n6.
Update Register Number\n7. Update Age\n8. Delete by Position\n9. Delete by Register
Number\n");
		printf("Enter Your Choice: ");
		scanf("%d",&choice);

		if (choice==0)
			break;

		switch (choice) {
			case 1:
				display();
				break;
			case 2:
				append();
				printf("Node Successfully Appended.\n");
				break;
			case 3:
				push();
				printf("Node Successfully Pushed.\n");
				break;
			case 4:
				printf("Choose the position where you wish to insert your new node
(0-%d): ",len);
				scanf("%d",&pos);
				if (pos<0 || pos>len) {
					printf("Invalid Choice.\n");
					continue;
				}
				insert(pos);
				printf("Node Successfully Inserted.\n");
				break;
			case 5:
```

```
                printf("Choose the position of the node whose name who wish to update
(0-%d): ",len-1);
                scanf("%d",&pos);
                if (pos<0 || pos>len-1) {
                        printf("Invalid Choice.\n");
                        continue;
                }
                updateName(pos);
                printf("Node Successfully Updated.\n");
                break;
            case 6:
                printf("Choose the position of the node whose register number who wish
to update (0-%d): ",len-1);
                scanf("%d",&pos);
                if (pos<0 || pos>len-1) {
                        printf("Invalid Choice.\n");
                        continue;
                }
                updateNo(pos);
                printf("Node Successfully Updated.\n");
                break;
            case 7:
                printf("Choose the position of the node whose age who wish to update
(0-%d): ",len-1);
                scanf("%d",&pos);
                if (pos<0 || pos>len-1) {
                        printf("Invalid Choice.\n");
                        continue;
                }
                updateAge(pos);
                printf("Node Successfully Updated.\n");
                break;
            case 8:
                printf("Choose the position of the node that you wish to delete (0-%d):
",len-1);
                scanf("%d",&pos);
                if (pos<0 || pos>len-1) {
                        printf("Invalid Choice.\n");
                        continue;
                }
                deleteByPosition(pos);
                printf("Node Successfully Deleted.\n");
                break;
            case 9:
                displayNo();
                printf("Choose the register number of the node that you wish to delete: ");
```

```
                scanf("%d",&pos);
                if (deleteByNo(pos))
                    printf("Node Successfully Deleted.\n");
                else
                    printf("Invalid Register Number.\n");
                break;

            default:
                printf("Invalid Choice.\n");

        }
    }
}
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/c_programs/exp_07$ cc linked_list.c
warspinix@Yuvaraj-Laptop:~/c_programs/exp_07$ ./a.out
Head Node Details
Name: BL
Register Number: 93
Age: 18

0. Exit
1. Display
2. Append
3. Push
4. Insert
5. Update Name
6. Update Register Number
7. Update Age
8. Delete by Position
9. Delete by Register Number
Enter Your Choice: 2
Name: Jamal
Register Number: 1
Age: 18
Node Successfully Appended.

0. Exit
1. Display
2. Append
3. Push
4. Insert
5. Update Name
6. Update Register Number
7. Update Age
8. Delete by Position
9. Delete by Register Number
Enter Your Choice: 3
Name: Yuvaraj
Register Number: 77
Age: 19
Node Successfully Pushed.
```

```
0. Exit
1. Display
2. Append
3. Push
4. Insert
5. Update Name
6. Update Register Number
7. Update Age
8. Delete by Position
9. Delete by Register Number
Enter Your Choice: 4
Choose the position where you wish to insert your new node (0-3): 0
Name: Krishnaa
Register Number: 104
Age: 18
Node Successfully Inserted.

0. Exit
1. Display
2. Append
3. Push
4. Insert
5. Update Name
6. Update Register Number
7. Update Age
8. Delete by Position
9. Delete by Register Number
Enter Your Choice: 4
Choose the position where you wish to insert your new node (0-4): 2
Name: Kabilesh
Register Number: 78
Age: 19
Node Successfully Inserted.
```

```
0. Exit
1. Display
2. Append
3. Push
4. Insert
5. Update Name
6. Update Register Number
7. Update Age
8. Delete by Position
9. Delete by Register Number
Enter Your Choice: 4
Choose the position where you wish to insert your new node (0-5): 5
Name: Nithish
Register Number: 99
Age: 18
Node Successfully Inserted.

0. Exit
1. Display
2. Append
3. Push
4. Insert
5. Update Name
6. Update Register Number
7. Update Age
8. Delete by Position
9. Delete by Register Number
Enter Your Choice: 1

Node    |Student Name   |No     |Age    |Address        |Next Address
Head    |Krishnaa       |104    |18     |0x55b7c1cd3b50 |0x55b7c1cd3b20
Node 1  |Yuvaraj        |77     |19     |0x55b7c1cd3b20 |0x55b7c1cd3b80
Node 2  |Kabilesh       |78     |19     |0x55b7c1cd3b80 |0x55b7c1cd32a0
Node 3  |BL             |93     |18     |0x55b7c1cd32a0 |0x55b7c1cd3af0
Node 4  |Jamal          |1      |18     |0x55b7c1cd3af0 |0x55b7c1cd3bb0
Node 5  |Nithish        |99     |18     |0x55b7c1cd3bb0 |NULL

Total Number of Nodes = 6
```

```
0. Exit
1. Display
2. Append
3. Push
4. Insert
5. Update Name
6. Update Register Number
7. Update Age
8. Delete by Position
9. Delete by Register Number
Enter Your Choice: 5
Choose the position of the node whose name who wish to update (0-5): 3
Old Name: BL
Updated Name: BK
Node Successfully Updated.

0. Exit
1. Display
2. Append
3. Push
4. Insert
5. Update Name
6. Update Register Number
7. Update Age
8. Delete by Position
9. Delete by Register Number
Enter Your Choice: 6
Choose the position of the node whose register number who wish to update (0-5): 1
Old Register Number: 77
Updated Register Number: 79
Node Successfully Updated.
```

```
0. Exit
1. Display
2. Append
3. Push
4. Insert
5. Update Name
6. Update Register Number
7. Update Age
8. Delete by Position
9. Delete by Register Number
Enter Your Choice: 7
Choose the position of the node whose age who wish to update (0-5): 4
Old Age: 18
Updated Age: 19
Node Successfully Updated.

0. Exit
1. Display
2. Append
3. Push
4. Insert
5. Update Name
6. Update Register Number
7. Update Age
8. Delete by Position
9. Delete by Register Number
Enter Your Choice: 1

Node    |Student Name   |No     |Age    |Address        |Next Address
Head    |Krishnaa       |104    |18     |0x55b7c1cd3b50 |0x55b7c1cd3b20
Node 1  |Yuvaraj        |79     |19     |0x55b7c1cd3b20 |0x55b7c1cd3b80
Node 2  |Kabilesh       |78     |19     |0x55b7c1cd3b80 |0x55b7c1cd32a0
Node 3  |BK             |93     |18     |0x55b7c1cd32a0 |0x55b7c1cd3af0
Node 4  |Jamal          |1      |19     |0x55b7c1cd3af0 |0x55b7c1cd3bb0
Node 5  |Nithish        |99     |18     |0x55b7c1cd3bb0 |NULL

Total Number of Nodes = 6
```

```
0. Exit
1. Display
2. Append
3. Push
4. Insert
5. Update Name
6. Update Register Number
7. Update Age
8. Delete by Position
9. Delete by Register Number
Enter Your Choice: 8
Choose the position of the node that you wish to delete (0-5): 0
Node Successfully Deleted.

0. Exit
1. Display
2. Append
3. Push
4. Insert
5. Update Name
6. Update Register Number
7. Update Age
8. Delete by Position
9. Delete by Register Number
Enter Your Choice: 8
Choose the position of the node that you wish to delete (0-4): 4
Node Successfully Deleted.
```

```
0. Exit
1. Display
2. Append
3. Push
4. Insert
5. Update Name
6. Update Register Number
7. Update Age
8. Delete by Position
9. Delete by Register Number
Enter Your Choice: 9

All Available Students
1) 79
2) 78
3) 93
4) 1
Choose the register number of the node that you wish to delete: 78
Node Successfully Deleted.

0. Exit
1. Display
2. Append
3. Push
4. Insert
5. Update Name
6. Update Register Number
7. Update Age
8. Delete by Position
9. Delete by Register Number
Enter Your Choice: 10
Invalid Choice.
```

```
0. Exit
1. Display
2. Append
3. Push
4. Insert
5. Update Name
6. Update Register Number
7. Update Age
8. Delete by Position
9. Delete by Register Number
Enter Your Choice: 0
warspinix@Yuvaraj-Laptop:~/c_programs/exp_07$
```

Evaluation Criteria	**Observation**	**Record**
Ability for problem definition and realisation	…../**10**	…../**10**
Ability to design and analyse	…../**10**	…../**10**
Ability to implement and validate	…../**10**	…../**10**

RESULT

The various operations of a linked list - creation of the head node, pushing, appending and inserting of new nodes, updating node data, and deleting of nodes by position and register number were all successfully implemented.

Exp. No: 8
Date: 23/11/23

IMPLEMENTATION OF STACK USING ARRAY AND LINKED LIST

AIM

To implement stack operations using array and linked list.

STACK USING ARRAY

```
#include <stdio.h>
#include <stdlib.h>

int *stack, top=-1, size;

void createStack() {
	stack=(int *)malloc(size*sizeof(int));
	stack_array=(int *)malloc(size*sizeof(int));
}

int isEmpty() {
	return top==-1;
}

int isFull() {
	return top==size-1;
}

void push (int element) {
	top++;
	stack[top]=element;
}

int pop() {
	top--;
	return stack[top+1];
}
```

```
int peek() {
	return stack[top];
}

int length() {
	return top+1;
}

void display() {
	int temp=top;
	printf("Elements of the Stack: ");
	while (temp!=-1) {
		printf("%d ",stack[temp]);
		temp--;
	}
	printf("\n");
}

int isPalindrome() {
	int l=top+1;
	int i, a[l];
	for (i=0;i<l;i++)
		a[i]=pop();
	top=l-1;
	int start=0,end=top;
	while (start<=end) {
		if (a[start]!=a[end])
			return 0;
		start++;
		end--;
	}
	return 1;
}

int main() {
	printf("Stack Implementation using Array\n\n");
	printf("Size = ");
	scanf("%d",&size);
	createStack();

	int choice, element;
```

```
    while(1) {
        printf("\n0. Exit\n1. Push\n2. Pop\n3. Peek\n4. Length\n5. Display All\n6. Check
Palindromic Stack\n");
        printf("Enter Your Choice: ");
        scanf("%d",&choice);

        if (choice==0)
            break;

        switch(choice) {
            case 1:
                if (isFull())
                    printf("Stack is full.\n");
                else {
                    printf("Element to be pushed: ");
                    scanf("%d",&element);
                    push(element);
                    printf("%d pushed onto stack.\n",element);
                }
                break;
            case 2:
                if (isEmpty())
                    printf("Stack is empty.\n");
                else
                    printf("Popped Element: %d\n",pop());
                break;
            case 3:
                if (isEmpty())
                    printf("Stack is empty.\n");
                else
                    printf("Top of the Stack: %d\n",peek());
                break;
            case 4:
                printf("No. of Stack Elements = %d\n",length());
                break;
            case 5:
                if (isEmpty())
                    printf("Stack is empty.\n");
                else
                    display();
                break;
            case 6:
                if (isEmpty())
                    printf("Stack is empty.\n");
                else if (isPalindrome())
                    printf("Stack is a Palindrome.\n");
```

```
                    else
                        printf("Stack is NOT a Palindrome.\n");
                    break;
                default:
                    printf("Invalid Choice.\n");
            }
        }
        printf("\nEnd of Program.\n");
}
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/c_programs/exp_08$ cc stack_array.c
warspinix@Yuvaraj-Laptop:~/c_programs/exp_08$ ./a.out
Stack Implementation using Array

Size = 5

0. Exit
1. Push
2. Pop
3. Peek
4. Length
5. Display All
6. Check Palindromic Stack
Enter Your Choice: 2
Stack is empty.

0. Exit
1. Push
2. Pop
3. Peek
4. Length
5. Display All
6. Check Palindromic Stack
Enter Your Choice: 1
Element to be pushed: 1
1 pushed onto stack.

0. Exit
1. Push
2. Pop
3. Peek
4. Length
5. Display All
6. Check Palindromic Stack
Enter Your Choice: 1
Element to be pushed: 2
2 pushed onto stack.
```

```
0. Exit
1. Push
2. Pop
3. Peek
4. Length
5. Display All
6. Check Palindromic Stack
Enter Your Choice: 1
Element to be pushed: 3
3 pushed onto stack.

0. Exit
1. Push
2. Pop
3. Peek
4. Length
5. Display All
6. Check Palindromic Stack
Enter Your Choice: 1
Element to be pushed: 4
4 pushed onto stack.

0. Exit
1. Push
2. Pop
3. Peek
4. Length
5. Display All
6. Check Palindromic Stack
Enter Your Choice: 5
Elements of the Stack: 4 3 2 1
```

```
0. Exit
1. Push
2. Pop
3. Peek
4. Length
5. Display All
6. Check Palindromic Stack
Enter Your Choice: 2
Popped Element: 4

0. Exit
1. Push
2. Pop
3. Peek
4. Length
5. Display All
6. Check Palindromic Stack
Enter Your Choice: 1
Element to be pushed: 10
10 pushed onto stack.
```

```
0. Exit
1. Push
2. Pop
3. Peek
4. Length
5. Display All
6. Check Palindromic Stack
Enter Your Choice: 1
Element to be pushed: 9
9 pushed onto stack.

0. Exit
1. Push
2. Pop
3. Peek
4. Length
5. Display All
6. Check Palindromic Stack
Enter Your Choice: 5
Elements of the Stack: 9 10 3 2 1

0. Exit
1. Push
2. Pop
3. Peek
4. Length
5. Display All
6. Check Palindromic Stack
Enter Your Choice: 6
Stack is NOT a Palindrome.
```

```
0. Exit
1. Push
2. Pop
3. Peek
4. Length
5. Display All
6. Check Palindromic Stack
Enter Your Choice: 2
Popped Element: 9

0. Exit
1. Push
2. Pop
3. Peek
4. Length
5. Display All
6. Check Palindromic Stack
Enter Your Choice: 2
Popped Element: 10
```

```
0. Exit
1. Push
2. Pop
3. Peek
4. Length
5. Display All
6. Check Palindromic Stack
Enter Your Choice: 4
No. of Stack Elements = 3

0. Exit
1. Push
2. Pop
3. Peek
4. Length
5. Display All
6. Check Palindromic Stack
Enter Your Choice: 1
Element to be pushed: 2
2 pushed onto stack.

0. Exit
1. Push
2. Pop
3. Peek
4. Length
5. Display All
6. Check Palindromic Stack
Enter Your Choice: 1
Element to be pushed: 1
1 pushed onto stack.
```

```
0. Exit
1. Push
2. Pop
3. Peek
4. Length
5. Display All
6. Check Palindromic Stack
Enter Your Choice: 5
Elements of the Stack: 1 2 3 2 1

0. Exit
1. Push
2. Pop
3. Peek
4. Length
5. Display All
6. Check Palindromic Stack
Enter Your Choice: 6
Stack is a Palindrome.
```

```
0. Exit
1. Push
2. Pop
3. Peek
4. Length
5. Display All
6. Check Palindromic Stack
Enter Your Choice: 1
Stack is full.

0. Exit
1. Push
2. Pop
3. Peek
4. Length
5. Display All
6. Check Palindromic Stack
Enter Your Choice: 3
Top of the Stack: 1

0. Exit
1. Push
2. Pop
3. Peek
4. Length
5. Display All
6. Check Palindromic Stack
Enter Your Choice: 0

End of Program.
warspinix@Yuvaraj-Laptop:~/c_programs/exp_08$
```

STACK USING LINKED LIST

```
#include <stdio.h>
#include <stdlib.h>

int size, length=0;

struct node {
	int data;
	struct node * next;
};

struct node * head;
struct node **headRef=&head;
struct node * top;
struct node * last;

int isEmpty() {
	return *headRef==NULL;
}

int isFull() {
	return length==size;
}

void push(int element) {
	struct node * new=(struct node *)malloc(sizeof(struct node));
	new->data=element;
	new->next=*headRef;
	*headRef=new;
	length++;
}

int pop () {
	top=*headRef;
	int t=top->data;
	*headRef=top->next;
	free(top);
	length--;
	return t;
}

int peek() {
	last=*headRef;
	return last->data;
```

```
}

void display() {
	last=*headRef;
	printf("Elements of the Stack: ");
	while (last!=NULL) {
		printf("%d ",last->data);
		last=last->next;
	}
	printf("\n");
}

int isPalindrome() {
	int l=length;
	int i, a[l];
	for (i=0;i<l;i++)
		a[i]=pop();
	for (i=l-1;i>=0;i--)
		push(a[i]);
	int start=0, end=length-1;
	while (start<=end) {
		if (a[start]!=a[end])
			return 0;
		start++;
		end--;
	}
	return 1;
}

int main() {
	printf("Stack Implementation using Linked List\n\n");
	printf("Size = ");
	scanf("%d",&size);
	int choice, element;

	while(1) {
		printf("\n0. Exit\n1. Push\n2. Pop\n3. Peek\n4. Length\n5. Display All\n6) Check
Palindromic Stack\n");
		printf("Enter Your Choice: ");
		scanf("%d",&choice);

		if (choice==0)
			break;

		switch(choice) {
			 case 1:
```

```
                if (isFull())
                        printf("Stack is full.\n");
                else {
                        printf("Element to be pushed: ");
                        scanf("%d",&element);
                        push(element);
                        printf("%d pushed onto stack.\n",element);
                }
                break;
        case 2:
                if (isEmpty())
                        printf("Stack is empty.\n");
                else
                        printf("Popped Element: %d\n",pop());
                break;
        case 3:
                if (isEmpty())
                        printf("Stack is empty.\n");
                else
                        printf("Top of the Stack: %d\n",peek());
                break;
        case 4:
                printf("No. of Stack Elements = %d\n",length);
                break;
        case 5:
                if (isEmpty())
                        printf("Stack is empty.\n");
                else
                        display();
                break;
        case 6:
                if (isEmpty())
                        printf("Stack is empty.\n");
                else if (isPalindrome())
                        printf("Stack is a Palindrome.\n");
                else
                        printf("Stack is NOT a Palindrome.\n");
                break;

        default:
                printf("Invalid Choice.\n");
            }
        }
        printf("\nEnd of Program.\n");
}
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/c_programs/exp_08$ cc stack_ll.c
warspinix@Yuvaraj-Laptop:~/c_programs/exp_08$ ./a.out
Stack Implementation using Linked List

Size = 5

0. Exit
1. Push
2. Pop
3. Peek
4. Length
5. Display All
6) Check Palindromic Stack
Enter Your Choice: 2
Stack is empty.

0. Exit
1. Push
2. Pop
3. Peek
4. Length
5. Display All
6) Check Palindromic Stack
Enter Your Choice: 1
Element to be pushed: 1
1 pushed onto stack.

0. Exit
1. Push
2. Pop
3. Peek
4. Length
5. Display All
6) Check Palindromic Stack
Enter Your Choice: 1
Element to be pushed: 2
2 pushed onto stack.
```

```
0. Exit
1. Push
2. Pop
3. Peek
4. Length
5. Display All
6. Check Palindromic Stack
Enter Your Choice: 1
Element to be pushed: 3
3 pushed onto stack.

0. Exit
1. Push
2. Pop
3. Peek
4. Length
5. Display All
6. Check Palindromic Stack
Enter Your Choice: 1
Element to be pushed: 4
4 pushed onto stack.

0. Exit
1. Push
2. Pop
3. Peek
4. Length
5. Display All
6. Check Palindromic Stack
Enter Your Choice: 5
Elements of the Stack: 4 3 2 1
```

```
0. Exit
1. Push
2. Pop
3. Peek
4. Length
5. Display All
6. Check Palindromic Stack
Enter Your Choice: 2
Popped Element: 4

0. Exit
1. Push
2. Pop
3. Peek
4. Length
5. Display All
6. Check Palindromic Stack
Enter Your Choice: 1
Element to be pushed: 10
10 pushed onto stack.
```

```
0. Exit
1. Push
2. Pop
3. Peek
4. Length
5. Display All
6. Check Palindromic Stack
Enter Your Choice: 1
Element to be pushed: 9
9 pushed onto stack.

0. Exit
1. Push
2. Pop
3. Peek
4. Length
5. Display All
6. Check Palindromic Stack
Enter Your Choice: 5
Elements of the Stack: 9 10 3 2 1

0. Exit
1. Push
2. Pop
3. Peek
4. Length
5. Display All
6. Check Palindromic Stack
Enter Your Choice: 6
Stack is NOT a Palindrome.
```

```
0. Exit
1. Push
2. Pop
3. Peek
4. Length
5. Display All
6. Check Palindromic Stack
Enter Your Choice: 2
Popped Element: 9

0. Exit
1. Push
2. Pop
3. Peek
4. Length
5. Display All
6. Check Palindromic Stack
Enter Your Choice: 2
Popped Element: 10
```

```
0. Exit
1. Push
2. Pop
3. Peek
4. Length
5. Display All
6. Check Palindromic Stack
Enter Your Choice: 4
No. of Stack Elements = 3

0. Exit
1. Push
2. Pop
3. Peek
4. Length
5. Display All
6. Check Palindromic Stack
Enter Your Choice: 1
Element to be pushed: 2
2 pushed onto stack.

0. Exit
1. Push
2. Pop
3. Peek
4. Length
5. Display All
6. Check Palindromic Stack
Enter Your Choice: 1
Element to be pushed: 1
1 pushed onto stack.
```

```
0. Exit
1. Push
2. Pop
3. Peek
4. Length
5. Display All
6. Check Palindromic Stack
Enter Your Choice: 5
Elements of the Stack: 1 2 3 2 1

0. Exit
1. Push
2. Pop
3. Peek
4. Length
5. Display All
6. Check Palindromic Stack
Enter Your Choice: 6
Stack is a Palindrome.
```

```
0. Exit
1. Push
2. Pop
3. Peek
4. Length
5. Display All
6. Check Palindromic Stack
Enter Your Choice: 1
Stack is full.

0. Exit
1. Push
2. Pop
3. Peek
4. Length
5. Display All
6. Check Palindromic Stack
Enter Your Choice: 3
Top of the Stack: 1

0. Exit
1. Push
2. Pop
3. Peek
4. Length
5. Display All
6. Check Palindromic Stack
Enter Your Choice: 0

End of Program.
warspinix@Yuvaraj-Laptop:~/c_programs/exp_08$
```

Evaluation Criteria	Observation	Record
Ability for problem definition and realisation	…../**10**	…../**10**
Ability to design and analyse	…../**10**	…../**10**
Ability to implement and validate	…../**10**	…../**10**

RESULT

Implementation of stack was successfully done using array and linked list.

Exp. No: 9
Date: 30/11/23

IMPLEMENTATION OF QUEUE USING ARRAY AND LINKED LIST

AIM

To implement queue operations using array and linked list.

QUEUE USING ARRAY

```
#include <stdio.h>
#include <stdlib.h>

int *queue, front=-1, back=-1, size;

void createQueue() {
	queue=(int *)malloc(size*sizeof(int));
}
int isEmpty() {
	return back==-1;
}
int isFull() {
	return back==size-1;
}

void enqueue (int element) {
	if (front==-1)
		front++;
	back++;
	queue[back]=element;
}

int dequeue () {
	int exited=queue[front];
	front++;
	if (front>back) {
		front=-1;
		back=-1;
	}
	return exited;
}
```

```
int peek() {
	return queue[front];
}

int length() {
	if (front==-1 && back==-1)
		return 0;
	return back-front+1;
}

void display() {
	int temp=front;
	printf("Elements of the Queue: ");
	while (temp<=back) {
		printf("%d ",queue[temp]);
		temp++;
	}
	printf("\n");
}

int main() {
	printf("Queue Implementation using Array\n\n");
	printf("Size = ");
	scanf("%d",&size);
	createQueue();

	int choice, element;

	while(1) {
		printf("\n0. Exit\n1. Enqueue\n2. Dequeue\n3. Peek\n4. Length\n5. Display All\n");
		printf("Enter Your Choice: ");
		scanf("%d",&choice);

		if (choice==0)
			break;

		switch(choice) {
			case 1:
				if (isFull())
					printf("Queue is full.\n");
				else {
					printf("Element to be enqueued: ");
					scanf("%d",&element);
					enqueue(element);
					printf("%d enqueued onto queue.\n",element);
```

```
                }
                break;
            case 2:
                if (isEmpty())
                    printf("Queue is empty.\n");
                else
                    printf("Dequeued Element: %d\n",dequeue());
                break;
            case 3:
                if (isEmpty())
                    printf("Queue is empty.\n");
                else
                    printf("Front of the Queue: %d\n",peek());
                break;
            case 4:
                printf("No. of Queue Elements = %d\n",length());
                break;
            case 5:
                if (isEmpty())
                    printf("Queue is empty.\n");
                else
                    display();
                break;
            default:
                printf("Invalid Choice.\n");
        }
    }
    printf("\nEnd of Program.\n");
}
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/c_programs/exp_09$ cc queue_array.c
warspinix@Yuvaraj-Laptop:~/c_programs/exp_09$ ./a.out
Queue Implementation using Array

Size = 4

0. Exit
1. Enqueue
2. Dequeue
3. Peek
4. Length
5. Display All
Enter Your Choice: 2
Queue is empty.

0. Exit
1. Enqueue
2. Dequeue
3. Peek
4. Length
5. Display All
Enter Your Choice: 1
Element to be enqueued: 1
1 enqueued onto queue.

0. Exit
1. Enqueue
2. Dequeue
3. Peek
4. Length
5. Display All
Enter Your Choice: 1
Element to be enqueued: 2
2 enqueued onto queue.
```

```
0. Exit
1. Enqueue
2. Dequeue
3. Peek
4. Length
5. Display All
Enter Your Choice: 1
Element to be enqueued: 3
3 enqueued onto queue.

0. Exit
1. Enqueue
2. Dequeue
3. Peek
4. Length
5. Display All
Enter Your Choice: 1
Element to be enqueued: 4
4 enqueued onto queue.

0. Exit
1. Enqueue
2. Dequeue
3. Peek
4. Length
5. Display All
Enter Your Choice: 1
Queue is full.
```

```
0. Exit
1. Enqueue
2. Dequeue
3. Peek
4. Length
5. Display All
Enter Your Choice: 2
Dequeued Element: 1

0. Exit
1. Enqueue
2. Dequeue
3. Peek
4. Length
5. Display All
Enter Your Choice: 2
Dequeued Element: 2
```

```
0. Exit
1. Enqueue
2. Dequeue
3. Peek
4. Length
5. Display All
Enter Your Choice: 1
Queue is full.

0. Exit
1. Enqueue
2. Dequeue
3. Peek
4. Length
5. Display All
Enter Your Choice: 3
Front of the Queue: 3

0. Exit
1. Enqueue
2. Dequeue
3. Peek
4. Length
5. Display All
Enter Your Choice: 4
No. of Queue Elements = 2
```

```
0. Exit
1. Enqueue
2. Dequeue
3. Peek
4. Length
5. Display All
Enter Your Choice: 2
Dequeued Element: 3

0. Exit
1. Enqueue
2. Dequeue
3. Peek
4. Length
5. Display All
Enter Your Choice: 1
Queue is full.
```

```
0. Exit
1. Enqueue
2. Dequeue
3. Peek
4. Length
5. Display All
Enter Your Choice: 2
Dequeued Element: 4

0. Exit
1. Enqueue
2. Dequeue
3. Peek
4. Length
5. Display All
Enter Your Choice: 5
Queue is empty.

0. Exit
1. Enqueue
2. Dequeue
3. Peek
4. Length
5. Display All
Enter Your Choice: 1
Element to be enqueued: 3
3 enqueued onto queue.

0. Exit
1. Enqueue
2. Dequeue
3. Peek
4. Length
5. Display All
Enter Your Choice: 0

End of Program.
warspinix@Yuvaraj-Laptop:~/c_programs/exp_09$
```

QUEUE USING LINKED LIST

```
#include <stdio.h>
#include <stdlib.h>

int size, queue_length=0, element_length=0;

struct node {
	int data;
	struct node * next;
};

struct node * f;
/*struct node * b;*/
void initialise() {
	f=(struct node *)malloc(sizeof(struct node));
	/*b=(struct node *)malloc(sizeof(struct node));*/
}

struct node **front=&f;
struct node * back;
struct node * temp;

int isEmpty() {
	return queue_length==0;
}

int isFull() {
	return queue_length==size;
}

void enqueue(int element) {
	if (isEmpty()) {
		initialise();
		back=*front;
		back->data=element;
	}
	else {
		struct node * new=(struct node *)malloc(sizeof(struct node));
		new->data=element;
		back->next=new;
		back=back->next;
	}
	queue_length++;
	element_length++;
```

```
}
int dequeue() {
	temp=*front;
	int t=temp->data;
	*front=temp->next;
	free(temp);
	element_length--;
	if (*front==NULL) {
		queue_length=0;
	}
	return t;
}

int peek() {
	temp=*front;
	return temp->data;
}

void display() {
	temp=*front;
	printf("Elements of the Queue: ");
	while (temp!=NULL) {
		printf("%d ",temp->data);
		temp=temp->next;
	}
	printf("\n");
}

int main() {
	printf("Queue Implementation using Linked List\n\n");
	printf("Size = ");
	scanf("%d",&size);
	int choice, element;

	while(1) {
		printf("\n0. Exit\n1. Enqueue\n2. Dequeue\n3. Peek\n4. Length\n5. Display All\n");
		printf("Enter Your Choice: ");
		scanf("%d",&choice);

		if (choice==0)
			break;

		switch(choice) {
			case 1:
				if (isFull())
					printf("Queue is full.\n");
```

```
                else {
                        printf("Element to be enqueued: ");
                        scanf("%d",&element);
                        enqueue(element);
                        printf("%d enqueued onto queue.\n",element);
                }
                break;
        case 2:
                if (isEmpty())
                        printf("Queue is empty.\n");
                else
                        printf("Dequeued Element: %d\n",dequeue());
                break;
        case 3:
                if (isEmpty())
                        printf("Queue is empty.\n");
                else
                        printf("Front of the Queue: %d\n",peek());
                break;
        case 4:
                printf("No. of Queue Elements = %d\n",element_length);
                break;
        case 5:
                if (isEmpty())
                        printf("Queue is empty.\n");
                else
                        display();
                break;
        default:
                printf("Invalid Choice.\n");
            }
        }
        printf("\nEnd of Program.\n");
}
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/c_programs/exp_09$ cc queue_ll.c
warspinix@Yuvaraj-Laptop:~/c_programs/exp_09$ ./a.out
Queue Implementation using Linked List

Size = 4

0. Exit
1. Enqueue
2. Dequeue
3. Peek
4. Length
5. Display All
Enter Your Choice: 2
Queue is empty.

0. Exit
1. Enqueue
2. Dequeue
3. Peek
4. Length
5. Display All
Enter Your Choice: 1
Element to be enqueued: 1
1 enqueued onto queue.

0. Exit
1. Enqueue
2. Dequeue
3. Peek
4. Length
5. Display All
Enter Your Choice: 1
Element to be enqueued: 2
2 enqueued onto queue.
```

```
0. Exit
1. Enqueue
2. Dequeue
3. Peek
4. Length
5. Display All
Enter Your Choice: 1
Element to be enqueued: 3
3 enqueued onto queue.

0. Exit
1. Enqueue
2. Dequeue
3. Peek
4. Length
5. Display All
Enter Your Choice: 1
Element to be enqueued: 4
4 enqueued onto queue.

0. Exit
1. Enqueue
2. Dequeue
3. Peek
4. Length
5. Display All
Enter Your Choice: 1
Queue is full.
```

```
0. Exit
1. Enqueue
2. Dequeue
3. Peek
4. Length
5. Display All
Enter Your Choice: 2
Dequeued Element: 1

0. Exit
1. Enqueue
2. Dequeue
3. Peek
4. Length
5. Display All
Enter Your Choice: 2
Dequeued Element: 2
```

```
0. Exit
1. Enqueue
2. Dequeue
3. Peek
4. Length
5. Display All
Enter Your Choice: 1
Queue is full.

0. Exit
1. Enqueue
2. Dequeue
3. Peek
4. Length
5. Display All
Enter Your Choice: 3
Front of the Queue: 3

0. Exit
1. Enqueue
2. Dequeue
3. Peek
4. Length
5. Display All
Enter Your Choice: 4
No. of Queue Elements = 2
```

```
0. Exit
1. Enqueue
2. Dequeue
3. Peek
4. Length
5. Display All
Enter Your Choice: 2
Dequeued Element: 3

0. Exit
1. Enqueue
2. Dequeue
3. Peek
4. Length
5. Display All
Enter Your Choice: 1
Queue is full.
```

```
0. Exit
1. Enqueue
2. Dequeue
3. Peek
4. Length
5. Display All
Enter Your Choice: 2
Dequeued Element: 4

0. Exit
1. Enqueue
2. Dequeue
3. Peek
4. Length
5. Display All
Enter Your Choice: 5
Queue is empty.

0. Exit
1. Enqueue
2. Dequeue
3. Peek
4. Length
5. Display All
Enter Your Choice: 1
Element to be enqueued: 3
3 enqueued onto queue.

0. Exit
1. Enqueue
2. Dequeue
3. Peek
4. Length
5. Display All
Enter Your Choice: 0

End of Program.
warspinix@Yuvaraj-Laptop:~/c_programs/exp_09$
```

Evaluation Criteria	**Observation**	**Record**
Ability for problem definition and realisation	…../**10**	…../**10**
Ability to design and analyse	…../**10**	…../**10**
Ability to implement and validate	…../**10**	…../**10**

RESULT

Implementation of queue was successfully done using array and linked list.

Exp. No: 10
Date: 14/12/23

IMPLEMENTATION OF BINARY SEARCH TREE AND TREE TRAVERSALS

AIM

To implement binary search tree, and perform tree traversals – inorder, preorder, and postorder.

BINARY SEARCH TREE CREATION

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
	int value;
	struct Node * left;
	struct Node * right;
};

struct Node * addNode(int val) {
	struct Node * new=(struct Node *)malloc(sizeof(struct Node));
	new->value=val;
	return new;
}

struct Node * insert (struct Node * root, int val) {
	if (root==NULL) {
		root=addNode(val);
		return root;
	}

	if (val<root->value)
		root->left=insert(root->left, val);
	else if (val>root->value)
		root->right=insert(root->right,val);

	return root;
}
```

SEARCH

```
struct Node * search(struct Node * root, int val) {
	if (root==NULL || root->value==val)
		return root;
	if (val<root->value)
		search(root->left,val);
	else
		search(root->right,val);
}
```

FIND INORDER SUCCESSOR

```
struct node * findInorderSuccessor(struct node * root) {
	if (root==NULL)
		return root;
	if (root->left!=NULL)
		return findInorderSuccessor(root->left);
	return root;
}
```

DELETE

```
struct node * delete(struct node * root, int val) {
	if (root==NULL)
		return root;
	if (root->value<val) {
		root->right=delete(root->right, val);
		return root;
	} else if (root->value>val) {
		root->left=delete(root->left, val);
		return root;
	}

	if (root->left==NULL && root->right==NULL) {
		free(root);
		return NULL;
	} else if (root->left==NULL) {
		struct node * temp=root->right;
		free(root);
		return temp;
	} else if (root->right==NULL) {
		struct node * temp=root->left;
		free(root);
		return temp;
```

```
	} else {
		struct node * inorderSuccessor=findInorderSuccessor(root->right);
		root->value=inorderSuccessor->value;
		root->right=delete(root->right,inorderSuccessor->value);
		return root;
	}
}
```

INORDER TRAVERSAL

```
void inorderTraversal(struct node * root) {
	if (root==NULL) return;
	inorderTraversal(root->left);
	printf("%d ", root->value);
	inorderTraversal(root->right);
}
```

PREORDER TRAVERSAL

```
void preorderTraversal(struct node * root) {
	if (root==NULL) return;
	printf("%d ",root->value);
	preorderTraversal(root->left);
	preorderTraversal(root->right);
}
```

POSTORDER TRAVERSAL

```
void postorderTraversal(struct node * root) {
	if (root==NULL) return;
	postorderTraversal(root->left);
	postorderTraversal(root->right);
	printf("%d ",root->value);
}
```

MAIN PROGRAM

```
int main() {
	struct node * root;
	int choice, val;

	printf("Binary Search Tree\n");
	while (1) {
		printf("\n0. Exit\n1. Insert\n2. Search\n3. Delete\n4. Inorder Traversal\n5. Preorder
Traversal\n6. Postorder Traversal");
		printf("\nYour Choice: ");
		scanf("%d",&choice);

		if (choice==0)
			break;

		switch (choice) {
			case 1:
				printf("\nValue to be Inserted: ");
				scanf("%d",&val);
				if (search(root, val)!=NULL)
					printf("Duplicates are not allowed.\n");
				else {
					if (root==NULL)
						root=insert(root, val);
					else
						insert(root, val);
					printf("%d has been successfully added to the binary search
tree.\n",val);
				}
				break;
			case 2:
				if (root==NULL)
					printf("\nThe tree is empty.\n");
				else {
					printf("\nValue to be Searched: ");
					scanf("%d",&val);
					if (search(root,val)!=NULL)
						printf("%d is present in the binary search tree.\n",val);
					else
						printf("%d is NOT present in the binary search tree.\n",val);
				}
				break;
			case 3:
				if (root==NULL)
```

```
                        printf("\nThe tree is empty.\n");
                    else {
                        printf("\nValue to be Deleted: ");
                        scanf("%d",&val);
                        if (search(root,val)!=NULL) {
                            root=delete(root,val);
                            printf("%d has been successfully deleted from the binary search
tree.\n",val);
                        } else
                            printf("%d is NOT present in the binary search tree.\n",val);
                    }
                    break;
                case 4:
                    if (root==NULL)
                        printf("\nThe tree is empty.\n");
                    else {
                        printf("\nInorder Traversal\n");
                        inorderTraversal(root);
                        printf("\n");
                    }
                    break;
                case 5:
                    if (root==NULL)
                        printf("\nThe tree is empty.\n");
                    else {
                        printf("\nPreorder Traversal\n");
                        preorderTraversal(root);
                        printf("\n");
                    }
                    break;
                case 6:
                    if (root==NULL)
                        printf("\nThe tree is empty.\n");
                    else {
                        printf("\nPostorder Traversal\n");
                        postorderTraversal(root);
                        printf("\n");
                    }
                    break;
                default:
                    printf("\nInvalid Choice.\n");
                    break;
            }
        }
}
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/c_programs/exp_10$ cc bst.c
warspinix@Yuvaraj-Laptop:~/c_programs/exp_10$ ./a.out
Binary Search Tree

0. Exit
1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
Your Choice: 1

Value to be Inserted: 100
100 has been successfully added to the binary search tree.

0. Exit
1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
Your Choice: 1

Value to be Inserted: 150
150 has been successfully added to the binary search tree.

0. Exit
1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
Your Choice: 1

Value to be Inserted: 50
50 has been successfully added to the binary search tree.
```

```
0. Exit
1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
Your Choice: 1

Value to be Inserted: 25
25 has been successfully added to the binary search tree.

0. Exit
1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
Your Choice: 1

Value to be Inserted: 75
75 has been successfully added to the binary search tree.

0. Exit
1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
Your Choice: 1

Value to be Inserted: 150
Duplicates are not allowed.
```

```
0. Exit
1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
Your Choice: 1

Value to be Inserted: 125
125 has been successfully added to the binary search tree.

0. Exit
1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
Your Choice: 1

Value to be Inserted: 175
175 has been successfully added to the binary search tree.

0. Exit
1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
Your Choice: 4

Inorder Traversal
25 50 75 100 125 150 175
```

```
0. Exit
1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
Your Choice: 5

Preorder Traversal
100 50 25 75 150 125 175

0. Exit
1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
Your Choice: 6

Postorder Traversal
25 75 50 125 175 150 100

0. Exit
1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
Your Choice: 2

Value to be Searched: 200
200 is NOT present in the binary search tree.
```

```
0. Exit
1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
Your Choice: 2

Value to be Searched: 125
125 is present in the binary search tree.

0. Exit
1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
Your Choice: 3

Value to be Deleted: 125
125 has been successfully deleted from the binary search tree.

0. Exit
1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
Your Choice: 3

Value to be Deleted: 100
100 has been successfully deleted from the binary search tree.
```

```
0. Exit
1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
Your Choice: 3

Value to be Deleted: 25
25 has been successfully deleted from the binary search tree.

0. Exit
1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
Your Choice: 4

Inorder Traversal
50 75 150 175

0. Exit
1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
Your Choice: 0
warspinix@Yuvaraj-Laptop:~/c_programs/exp_10$
```

Evaluation Criteria	**Observation**	**Record**
Ability for problem definition and realisation	…../**10**	…../**10**
Ability to design and analyse	…../**10**	…../**10**
Ability to implement and validate	…../**10**	…../**10**

RESULT

Binary search tree was successfully implemented, and inorder, preorder, and postorder traversals were performed.

Exp. No: 11
Date: 21/12/23

IMPLEMENTATION OF HEAP SORT

AIM

To implement sorting using heap sort technique.

PRINT ARRAY

```
#include <stdio.h>

void print(int a[], int n) {
	int i;
	for (i=0;i<n;i++)
		printf("%d ",a[i]);
}
```

SWAP

```
void swap(int *a, int *b) {
	int temp=*a;
	*a=*b;
	*b=temp;
}
```

MIN HEAP – SORT IN DESCENDING ORDER

```
void minHeapify(int a[], int n, int i) {
	int smallest=i;
	int left=2*i+1;
	int right=2*i+2;
	if (left<n && a[left]<a[smallest])
		smallest=left;
	if (right<n && a[right]<a[smallest])
		smallest=right;
	if (smallest!=i) {
		swap(&a[i],&a[smallest]);
		minHeapify(a,n,smallest);
	}
}
```

```
void minHeapSort(int a[], int n) {
	int i;
	for (i=(n/2)-1;i>=0;i--)
		minHeapify(a,n,i);
	for (i=n-1;i>=0;i--) {
		swap(&a[0],&a[i]);
		minHeapify(a,i,0);
	}
}
```

MAX HEAP – SORT IN ASCENDING ORDER

```
void maxHeapify(int a[], int n, int i) {
	int largest=i;
	int left=2*i+1;
	int right=2*i+2;
	if (left<n && a[left]>a[largest])
		largest=left;
	if (right<n && a[right]>a[largest])
		largest=right;
	if (largest!=i) {
		swap(&a[i],&a[largest]);
		maxHeapify(a,n,largest);
	}
}

void maxHeapSort(int a[], int n) {
	int i,j;
	for (i=(n/2)-1;i>=0;i--) {
		maxHeapify(a,n,i);
	}
	for (i=n-1;i>=0;i--) {
		swap(&a[0],&a[i]);
		counter--;
		printf("Current Heap: ");
		for (j=0;j<counter;j++)
			printf("%d ",a[j]);
		printf("\n");
		printf("Current Array: ");
		for (j=counter;j<n;j++)
			printf("%d ",a[j]);
		printf("\n\n");
		maxHeapify(a,i,0);
	}
}
```

MAIN PROGRAM

```
int main() {
	int n, i;
	printf("No. of Elements: ");
	scanf("%d",&n);
	counter=n;
	int a[n];
	for (i=0;i<n;i++) {
		printf("Element %d: ",i+1);
		scanf("%d",&a[i]);
	}
	printf("\nArray Elements\n");
	print(a,n);
	printf("\n\nIntermediate Stages\n");
	maxHeapSort(a,n);
	printf("Sorted Elements in Ascending Order\n");
	print(a,n);
	printf("\n\nSorted Elements in Descending Order\n");
	minHeapSort(a,n);
	print(a,n);
	printf("\n");
}
```

OUTPUT

```
warspinix@Yuvaraj-Laptop:~/c_programs/exp_11$ cc heap_sort.c
warspinix@Yuvaraj-Laptop:~/c_programs/exp_11$ ./a.out
No. of Elements: 13
Element 1: 980
Element 2: 20
Element 3: 30
Element 4: 50
Element 5: 380
Element 6: 540
Element 7: 1020
Element 8: 850
Element 9: 700
Element 10: 5
Element 11: 10
Element 12: 100
Element 13: 1

Array Elements
980 20 30 50 380 540 1020 850 700 5 10 100 1
```

```
Intermediate Stages
Current Heap: 1 850 980 700 380 540 30 50 20 5 10 100
Current Array: 1020

Current Heap: 1 850 540 700 380 100 30 50 20 5 10
Current Array: 980 1020

Current Heap: 10 700 540 50 380 100 30 1 20 5
Current Array: 850 980 1020

Current Heap: 5 380 540 50 10 100 30 1 20
Current Array: 700 850 980 1020

Current Heap: 20 380 100 50 10 5 30 1
Current Array: 540 700 850 980 1020

Current Heap: 1 50 100 20 10 5 30
Current Array: 380 540 700 850 980 1020

Current Heap: 1 50 30 20 10 5
Current Array: 100 380 540 700 850 980 1020

Current Heap: 5 20 30 1 10
Current Array: 50 100 380 540 700 850 980 1020
```

```
Current Heap: 10 20 5 1
Current Array: 30 50 100 380 540 700 850 980 1020

Current Heap: 1 10 5
Current Array: 20 30 50 100 380 540 700 850 980 1020

Current Heap: 5 1
Current Array: 10 20 30 50 100 380 540 700 850 980 1020

Current Heap: 1
Current Array: 5 10 20 30 50 100 380 540 700 850 980 1020

Current Heap:
Current Array: 1 5 10 20 30 50 100 380 540 700 850 980 1020

Sorted Elements in Ascending Order
1 5 10 20 30 50 100 380 540 700 850 980 1020

Sorted Elements in Descending Order
1020 980 850 700 540 380 100 50 30 20 10 5 1
warspinix@Yuvaraj-Laptop:~/c_programs/exp_11$
```

Evaluation Criteria	Observation	Record
Ability for problem definition and realisation	…../**10**	…../**10**
Ability to design and analyse	…../**10**	…../**10**
Ability to implement and validate	…../**10**	…../**10**

RESULT

The given elements were successfully sorted in ascending and descending order using heap sort.